

Proyecto II: Series de Tiempo

Integrantes del Equipo 5:

- ☐ Omar Misael Valero Silva
- ☐ Karla Anahí Sánchez Morales
- ☐ Karol Jacqueline Nolasco Reyes
- ☐ David Mendieta Lopez
- ☐ César Gabriel López López

Profesor: LEONARDO ROMÁN

23 de Mayo del 2025

Parte Práctica

1. El archivo **Tasa de vida y muerte** es un conjunto de datos de las tendencias de mortalidad en Estados Unidos desde 1900 destaca las diferencias en las tasas de mortalidad ajustadas por edad y la esperanza de vida al nacer por raza y sexo. Las tasas de mortalidad ajustadas por edad (*muertes por 100.000*) después de 1998 se calculan con base en la población estándar de Estados Unidos del año 2000.

Las poblaciones utilizadas para calcular las tasas de mortalidad para 2011-2017 son estimaciones postcensales basadas en el censo de 2010, estimado al 1 de julio de 2010. Las tasas para los años del censo se basan en las poblaciones enumeradas en los censos correspondientes. Las tasas para los años sin censo entre 2000 y 2010 se revisan utilizando estimaciones de población intercensales actualizadas y pueden diferir de las tasas publicadas anteriormente.

Los datos sobre las tasas de mortalidad ajustadas por edad antes de 1999 se han tomado de datos históricos (Referencias a continuación). Los datos sobre la esperanza de vida están disponibles hasta 2017. Debido a los cambios en las categorías de raza utilizadas en las publicaciones, los datos para la población negra no están disponibles de manera consistente antes de 1968, y en absoluto antes de 1960.

Más información sobre datos históricos sobre tasas de mortalidad ajustadas por edad está disponible en:

<https://www.cdc.gov/nchs/nvss/mortality/hist293.html>.

FUENTES:

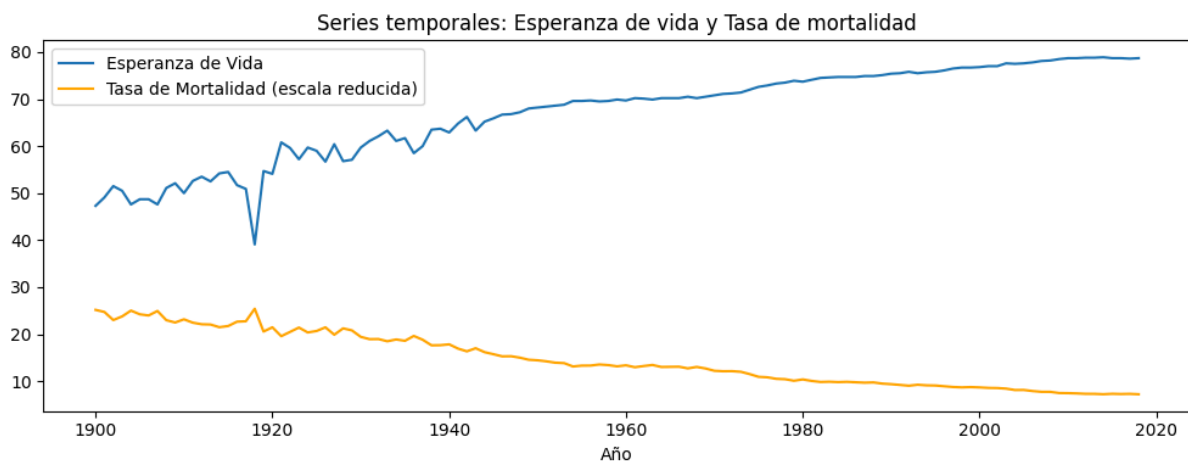
- CDC/NCHS, Sistema Nacional de Estadísticas Vitales, datos históricos, 1900-1998 (ver https://www.cdc.gov/nchs/nvss/mortality_historical_data.html).

- CDC/NCHS, Sistema Nacional de Estadísticas Vitales, datos de mortalidad (ver <http://www.cdc.gov/nchs/deaths.html>).
 - CDC WONDER (ver <http://wonder.cdc.gov>).
- a.** Estudie la base y modele, al menos un par de series temporales para cada segmento (los que hayan elegido). Valide ampliamente el análisis de cada modelo.

Estudiaremos la base de datos filtrando el segmento All Races - Both Sexes (toda la población de EE.UU., ambos sexos). Este segmento contiene dos series anuales de 1900 a 2017: la esperanza de vida al nacer (en años) y la tasa de mortalidad ajustada por edad (muertes anuales por 100,000 habitantes, ajustada a la población estándar).

Primero cargamos los datos y visualizamos ambas series para identificar tendencias o patrones:

```
15 data = pd.read_csv('Tasa de vida y muerte.csv')
16 data = data[(data['Race'] == 'All Races') & (data['Sex'] == 'Both Sexes')].copy()
17 data = data.sort_values(['Year'])
18
19 years = data['Year'].values
20 life_exp = data['Average Life Expectancy (Years)'].values
21 mort_rate = data['Age-adjusted Death Rate'].str.replace(',', '').astype(float).values
22
23
24 plt.figure(figsize=(10, 4))
25 plt.plot(years, life_exp, label='Esperanza de Vida')
26 plt.plot(years, mort_rate / 100, label='Tasa de Mortalidad (escala reducida)', color='orange')
27 plt.title('Series temporales: Esperanza de vida y Tasa de mortalidad')
28 plt.xlabel('Año')
29 plt.legend()
30 plt.tight_layout()
31 plt.show()
32
```



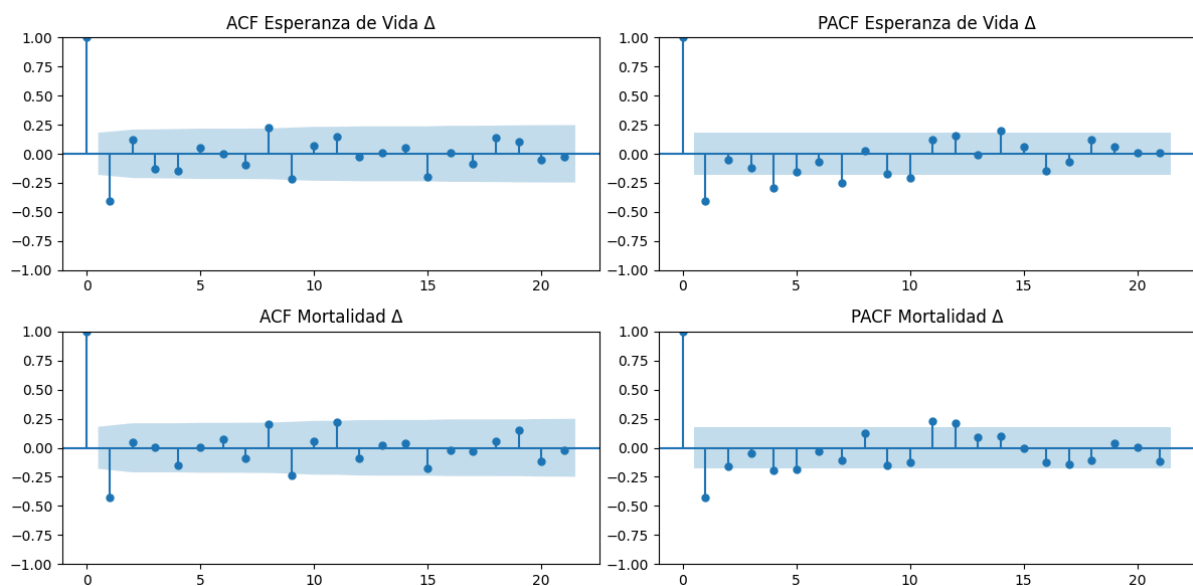
A simple vista, la esperanza de vida tiene un comportamiento creciente, por otro lado, la mortalidad exhibe un comportamiento decreciente. La esperanza de vida presenta incrementos graduales a lo largo del siglo XX (con estancamientos y retrocesos temporales en años específicos, e.g. 1918), mientras que la mortalidad muestra disminuciones pronunciadas especialmente en la primera mitad del siglo XX.

A continuación, examinamos los correlogramas de autocorrelación para las series diferenciadas con el fin de identificar posibles órdenes p (autoregresivos) y q (de medias móviles).

```

34 def adf_test(series, name):
35     result = adfuller(series, autolag='AIC')
36     print(f"ADF para {name}: p-value = {result[1]:.4f}")
37
38 adf_test(life_exp, 'Esperanza de Vida (nivel)')
39 adf_test(mort_rate, 'Mortalidad (nivel)')
40
41 dlife_exp = np.diff(life_exp)
42 dmort_rate = np.diff(mort_rate)
43
44 adf_test(dlife_exp, 'Esperanza de Vida (1ª diferencia)')
45 adf_test(dmort_rate, 'Mortalidad (1ª diferencia)')
46
47
48 fig, ax = plt.subplots(2, 2, figsize=(12, 6))
49 plot_acf(dlife_exp, ax=ax[0, 0])
50 plot_pacf(dlife_exp, ax=ax[0, 1])
51 plot_acf(dmort_rate, ax=ax[1, 0])
52 plot_pacf(dmort_rate, ax=ax[1, 1])
53 ax[0, 0].set_title('ACF Esperanza de Vida Δ')
54 ax[0, 1].set_title('PACF Esperanza de Vida Δ')
55 ax[1, 0].set_title('ACF Mortalidad Δ')
56 ax[1, 1].set_title('PACF Mortalidad Δ')
57 plt.tight_layout()
58 plt.show()
59

```



En ambos casos, la ACF de la serie diferenciada presenta un valor negativo significativo en lag 1, mientras que el PACF muestra también una cut-off temprana. Esta forma de los correlogramas es consistente con un modelo ARIMA(0,1,1) (modelo de media móvil de primer orden sobre la serie integrada de orden 1) o alternatively un ARIMA(1,1,0), ya que un MA(1) con coeficiente negativo o un AR(1) con coeficiente negativo pueden explicar la autocorrelación instantánea inversa observada. Para determinar la mejor especificación, probamos varios modelos candidatos (ARIMA(p,1,q)) y comparamos sus criterios de información (AIC):

```

61 models = {}
62 for p in [0, 1, 2]:
63     for q in [0, 1, 2]:
64         for trend in ['n', 'c']:
65             try:
66                 le_model = sm.tsa.SARIMAX(life_exp, order=(p, 1, q), trend=trend).fit(dis=False)
67                 models[(p, q, trend, 'LE')] = le_model.aic
68             except:
69                 continue
70
71 best_le = min([k for k in models if k[3] == 'LE'], key=lambda k: models[k])
72 print(f"Mejor modelo Esperanza de Vida: ARIMA({best_le[0]},{1},{best_le[1]}) con trend='{best_le[2]}' , AIC={models[best_le]:.2f}")
73
74 for p in [0, 1, 2]:
75     for q in [0, 1, 2]:
76         for trend in ['n', 'c']:
77             try:
78                 m_model = sm.tsa.SARIMAX(mort_rate, order=(p, 1, q), trend=trend).fit(dis=False)
79                 models[(p, q, trend, 'M')] = m_model.aic
80             except:
81                 continue
82
83 best_m = min([k for k in models if k[3] == 'M'], key=lambda k: models[k])
84 print(f"Mejor modelo Mortalidad: ARIMA({best_m[0]},{1},{best_m[1]}) con trend='{best_m[2]}' , AIC={models[best_m]:.2f}")
85

```

Mejor modelo Esperanza de Vida: ARIMA(1,1,1) con trend='c', AIC=501.61
Mejor modelo Mortalidad: ARIMA(0,1,1) con trend='c', AIC=1327.87

Esperanza de vida: un modelo ARIMA(1,1,1) con término constante tras la diferenciación. Este modelo tuvo el AIC más bajo, mejorando sobre ARIMA(0,1,1) y sobre alternativas ARIMA(1,1,0) o ARIMA(1,1,1) sin término constante.

Tasa de mortalidad: modelo ARIMA(0,1,1) con término constante igualmente resultó óptimo en AIC, capturando la tendencia descendente con un término constante (drift negativo) y la autocorrelación de corto plazo con un término MA(1).

Procedemos a estimar dichos modelos finales y revisar los parámetros estimados y diagnósticos de residuos:

```

2 model_le = sm.tsa.SARIMAX(life_exp, order=(best_le[0], 1, best_le[1]), trend=best_le[2]).fit()
3 model_m = sm.tsa.SARIMAX(mort_rate, order=(best_m[0], 1, best_m[1]), trend=best_m[2]).fit()
4
5 print(model_le.summary())
6 print(model_m.summary())

```

```

SARIMAX Results
=====
Dep. Variable:          y      No. Observations:          119
Model:                  SARIMAX(1, 1, 1)      Log Likelihood          -246.803
Date:                   Thu, 29 May 2025      AIC                      501.607
Time:                   05:14:38              BIC                      512.690
Sample:                 0      HQIC                      506.107
                        - 119
Covariance Type:        opg
=====
              coef      std err          z      P>|z|      [0.025      0.975]
-----
intercept      0.1872      0.043       4.315      0.000       0.102       0.272
ar.L1          0.2953      0.085       3.464      0.001       0.128       0.462
ma.L1         -0.8247      0.082      -10.037      0.000      -0.986      -0.664
sigma2         3.8172      0.180      21.229      0.000       3.465       4.170
=====
Ljung-Box (L1) (Q):           0.13      Jarque-Bera (JB):          2124.66
Prob(Q):                     0.72      Prob(JB):              0.00
Heteroskedasticity (H):       0.02      Skew:                  -1.66
Prob(H) (two-sided):         0.00      Kurtosis:              23.52
=====

```

```

SARIMAX Results
=====
Dep. Variable:          y      No. Observations:          119
Model:                  SARIMAX(0, 1, 1)      Log Likelihood          -660.935
Date:                   Thu, 29 May 2025      AIC                      1327.871
Time:                   05:14:38              BIC                      1336.183
Sample:                 0      HQIC                      1331.246
                        - 119
Covariance Type:        opg
=====
              coef      std err          z      P>|z|      [0.025      0.975]
-----
intercept     -14.9759       2.955      -5.068      0.000      -20.767      -9.184
ma.L1         -0.5860       0.038     -15.238      0.000      -0.661      -0.511
sigma2        4282.8526     264.549     16.189      0.000     3764.346     4801.359
=====
Ljung-Box (L1) (Q):           0.20      Jarque-Bera (JB):          278.08
Prob(Q):                     0.66      Prob(JB):              0.00
Heteroskedasticity (H):       0.04      Skew:                   0.23
Prob(H) (two-sided):         0.00      Kurtosis:              10.51
=====

```

```

108 ljung_le = acorr_ljungbox(resid_le, lags=[10], return_df=True)
109 ljung_m = acorr_ljungbox(resid_m, lags=[10], return_df=True)
110
111 print("Ljung-Box Esperanza de Vida: p =", ljung_le['lb_pvalue'].values[0])
112 print("Ljung-Box Mortalidad: p =", ljung_m['lb_pvalue'].values[0])
113

```

```

Ljung-Box Esperanza de Vida: p = 0.9914648363739123
Ljung-Box Mortalidad: p = 0.9995201830599086

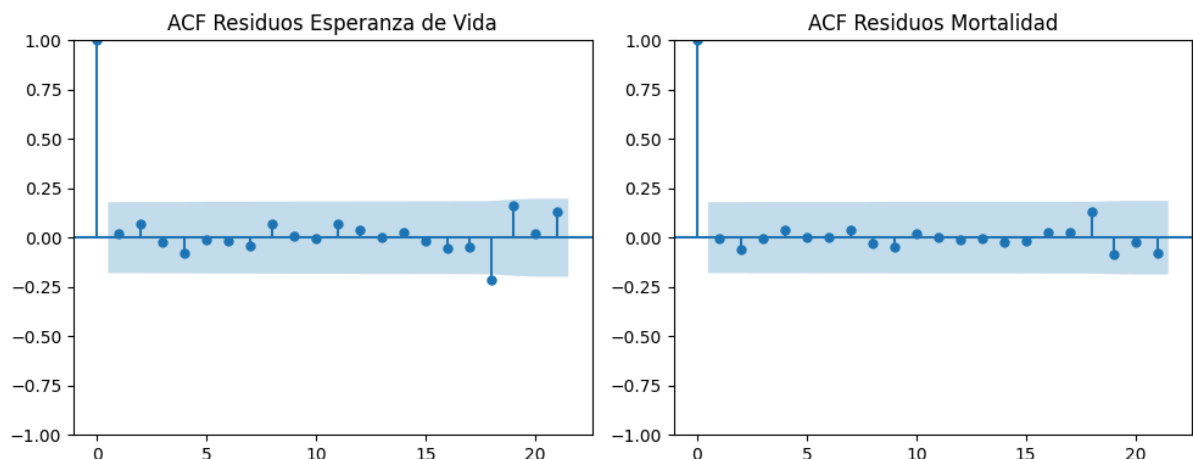
```

Validación de residuos: Ambos modelos muestran buen ajuste, sin autocorrelación remanente significativa. Los residuos (diferencias entre valores observados y ajustados por el modelo a un paso) están centrados en cero y no exhiben patrones sistemáticos. En particular, la prueba de Ljung-Box aplicada a 10 rezagos no rechaza la hipótesis de ruido blanco (p-valores > 0.05 en ambos casos), indicando que los residuos son esencialmente aleatorios. A continuación se grafican las autocorrelaciones de los residuos para confirmar visualmente la ausencia de estructura:

```

95 resid_le = model_le.resid
96 resid_m = model_m.resid
97
98
99 fig, ax = plt.subplots(1, 2, figsize=(10, 4))
100 plot_acf(resid_le, ax=ax[0])
101 plot_acf(resid_m, ax=ax[1])
102 ax[0].set_title('ACF Residuos Esperanza de Vida')
103 ax[1].set_title('ACF Residuos Mortalidad')
104 plt.tight_layout()
105 plt.show()
106

```



b. Saque una conclusión general de ambos segmentos.

Los análisis realizados nos permiten extraer varias conclusiones sobre la evolución de la esperanza de vida y la mortalidad en EE.UU.

- Tendencias de largo plazo:** La esperanza de vida al nacer tuvo un incremento dramático a lo largo del siglo XX y principios del XXI (aproximadamente +31 años en total). Este avance no fue lineal: fue particularmente rápido en la primera mitad del siglo XX, estabilizándose en torno a los 70 años desde 1950–1960, y alcanzando un máximo cerca de 78.9 años en 2014. Se observan pequeños descensos recientes (2015–2017) reflejando problemas de salud emergentes en la población. Por su parte, la tasa de mortalidad ajustada por edad mostró una disminución sostenida y muy pronunciada (reduciéndose a menos de la tercera parte de su valor de 1900). Gran parte de la caída ocurrió antes de 1950 (debido a mejoras sanitarias, médicas y en las condiciones de vida), con una desaceleración en la segunda mitad del siglo XX.

- **Fluctuaciones:** En ambas series se identifican fluctuaciones de corto plazo superpuestas a las tendencias. Destaca el año 1918, donde la esperanza de vida cae abruptamente y la mortalidad se eleva a un pico histórico (estos cambios extremos corresponden a la pandemia de gripe española, un evento externo que impactó fuertemente la demografía ese año). Este efecto se revierte en 1919 con un rebote significativo. Otros períodos con oscilaciones notables incluyen mediados de la década de 1940 (efecto de la Segunda Guerra Mundial y otros factores), fines de los 1950s (pandemia de gripe asiática), mediados de 1980s a 1990s (crisis del VIH/SIDA afectando la mortalidad) y mitad de la década de 2010 (ligera baja en esperanza de vida atribuida a factores como las sobredosis, enfermedades crónicas, etc.). No obstante, tras cada perturbación de corto plazo, ambas series retomaron su tendencia subyacente (aumento de longevidad, reducción de mortalidad).
- **Modelos ARIMA:** Los modelos ARIMA ajustados sugieren que, tras remover la tendencia (mediante diferenciación), las variaciones año a año tanto de la esperanza de vida como de la mortalidad tienen una memoria corta, básicamente de un año. En particular, un componente MA(1) negativo fue suficiente para explicar la autocorrelación: esto implica que un aumento anómalo en un año tiende a ser seguido por un pequeño descenso compensatorio al siguiente, y viceversa. Este fenómeno concuerda con la dinámica demográfica: por ejemplo, un año de mejora excepcional en esperanza de vida suele ir seguido por una mejora más moderada (o un leve retroceso) al siguiente año, y un año muy malo en mortalidad suele ser seguido por una corrección (mejora) al siguiente año.

c. Establezca el modelo matemático estimado en su forma original.

Esperanza de Vida – ARIMA(1,1,1) con constante

El modelo estimado para la serie de esperanza de vida corresponde a un ARIMA(1,1,1) con término constante (drift). La forma en diferencias es:

$$\Delta EV_t = c + \phi_1 \Delta EV_{t-1} + \theta_1 \varepsilon_{t-1} + \varepsilon_t$$

Sustituyendo los coeficientes estimados:

$$\Delta EV_t = \underbrace{0.1872}_c + \underbrace{0.2953}_{\phi_1} \Delta EV_{t-1} + \underbrace{-0.8247}_{\theta_1} \varepsilon_{t-1} + \varepsilon_t$$

En forma de nivel:

$$EV_t = EV_{t-1} + 0.1872 + 0.2953(EV_{t-1} - EV_{t-2}) - 0.8247\varepsilon_{t-1} + \varepsilon_t$$

Este modelo indica una tendencia promedio de crecimiento de 0.187 años por año, con dinámica de corto plazo representada por términos AR(1) y MA(1), y errores no correlacionados.

Tasa de Mortalidad – ARIMA(0,1,1) con constante

El modelo estimado para la serie de tasa de mortalidad ajustada por edad corresponde a un ARIMA(0,1,1) con término constante. La forma en diferencias es:

$$\Delta M_t = c' + \theta_1 \varepsilon_{t-1} + \varepsilon_t$$

Sustituyendo los coeficientes estimados:

$$\Delta M_t = \underbrace{-14.9759}_{c'} + \underbrace{-0.5860}_{\theta_1} \varepsilon_{t-1} + \varepsilon_t$$

En forma de nivel:

$$M_t = M_{t-1} - 14.9759 - 0.5860 \varepsilon_{t-1} + \varepsilon_t$$

Este modelo refleja una disminución promedio de aproximadamente 14.98 muertes por 100,000 habitantes por año, con un patrón de corrección rápida ante shocks de mortalidad del año previo.

Nota: En ambas ecuaciones, $\Delta EV_t = EV_t - EV_{t-1}$ y $\Delta M_t = M_t - M_{t-1}$. Los términos ε_t representan ruido blanco. Los coeficientes negativos de los componentes MA(1) indican una autocorrelación negativa de corto plazo, coherente con la estructura observada en los correlogramas.

2. El archivo “*API_FR.INR.LEND_DS2_es_excel_v2_3416697.xlsx*” contine las tasas de interés activa.
 - a. Modele, al menos un país de cada región (*Asia, Oriente Medio, Africa, Oceania, America, Europa*), incluyendo de América a México (*total 7 series, no olvide validar su modelo*).

Primero cargaremos el archivo para poder procesarlo e identificar las diferentes regiones:

```
[29] import pandas as pd

# Cargar el archivo Excel
archivo = "API_FR.INR.LEND_DS2_es_excel_v2_3416697.xlsx"

# Leer las hojas
data = pd.read_excel(archivo, sheet_name="Data")
meta_countries = pd.read_excel(archivo, sheet_name="Metadata - Countries")

[30] data = data.merge(meta_countries[['Country Code', 'Region']], on='Country Code', how='left')

# Verificar algunas regiones únicas
print(data['Region'].dropna().unique())
```

```
['Asia meridional' 'África al sur del Sahara (excluido altos ingresos)'
'Europa y Asia central (excluido altos ingresos)'
'América Latina y el Caribe (excluido altos ingresos)'
'Asia oriental y el Pacífico (excluido altos ingresos)'
'Orient Medio y Norte de África (excluido altos ingresos)']
```

Después obtendremos los países disponibles para cada región:


```

0 s # Contar cuántos datos hay por país
data['num_datos'] = data[columnas_anios].notna().sum(axis=1)

# Filtrar países con al menos 5 años de datos
países_con_datos = data[data['num_datos'] >= 5][['Country Name', 'Region', 'num_datos']]

# Ver los países disponibles por región
disponibles_por_region = países_con_datos.groupby('Region')['Country Name'].unique()

for region, países in disponibles_por_region.items():
    print(f"● {region}: {list(países)}")

```

● América Latina y el Caribe (excluido altos ingresos): ['Argentina', 'Belice', 'Bolivia', 'Brasil', 'Colombia', 'Costa Rica', 'Dominica', 'I
 ● Asia meridional: ['Afganistán', 'Bangladesh', 'Bhután', 'India', 'Sri Lanka', 'Maldivas', 'Pakistán']
 ● Asia oriental y el Pacífico (excluido altos ingresos): ['China', 'Fiji', 'Micronesia (Estados Federados de)', 'Indonesia', 'República Demo
 ● Europa y Asia central (excluido altos ingresos): ['Albania', 'Armenia', 'Azerbaiyán', 'Bosnia y Herzegovina', 'Belarus', 'Georgia', 'Kirgu
 ● Oriente Medio y Norte de África (excluido altos ingresos): ['Argelia', 'Egipto, República Árabe de', 'Irán, República Islámica del', 'Iraq
 ● África al sur del Sahara (excluido altos ingresos): ['Angola', 'Burundi', 'Benin', 'Burkina Faso', 'Botswana', "Côte d'Ivoire", 'Congo, Rep

Mostramos la lista seleccionada de países para cada región:

```

0 s # Lista final de países seleccionados (1 por región)
países_seleccionados = [
    "México",                # América Latina
    "India",                 # Asia Meridional
    "Filipinas",             # Asia Oriental / Pacífico
    "Ucrania",               # Europa / Asia Central
    "Egipto, República Árabe de", # Medio Oriente / África del Norte
    "Sudáfrica",             # África subsahariana
    "Fiji"                   # Oceanía (en Asia Pacífico)
]

series_seleccionadas = data[data["Country Name"].isin(países_seleccionados)]

# Verificar resultado
print("Series seleccionadas:")
print(series_seleccionadas[['Country Name', 'Indicator Name']])

```

Series seleccionadas:

	Country Name	Indicator Name
67	Egipto, República Árabe de	Tasa de interés activa (%)
76	Fiji	Tasa de interés activa (%)
109	India	Tasa de interés activa (%)
154	México	Tasa de interés activa (%)
187	Filipinas	Tasa de interés activa (%)
248	Ucrania	Tasa de interés activa (%)
263	Sudáfrica	Tasa de interés activa (%)

Con este código obtenemos la tasa de interés activa por cada país seleccionado:

```

1 s import matplotlib.pyplot as plt

anios = list(map(str, range(1960, 2024))) # Hasta 2023

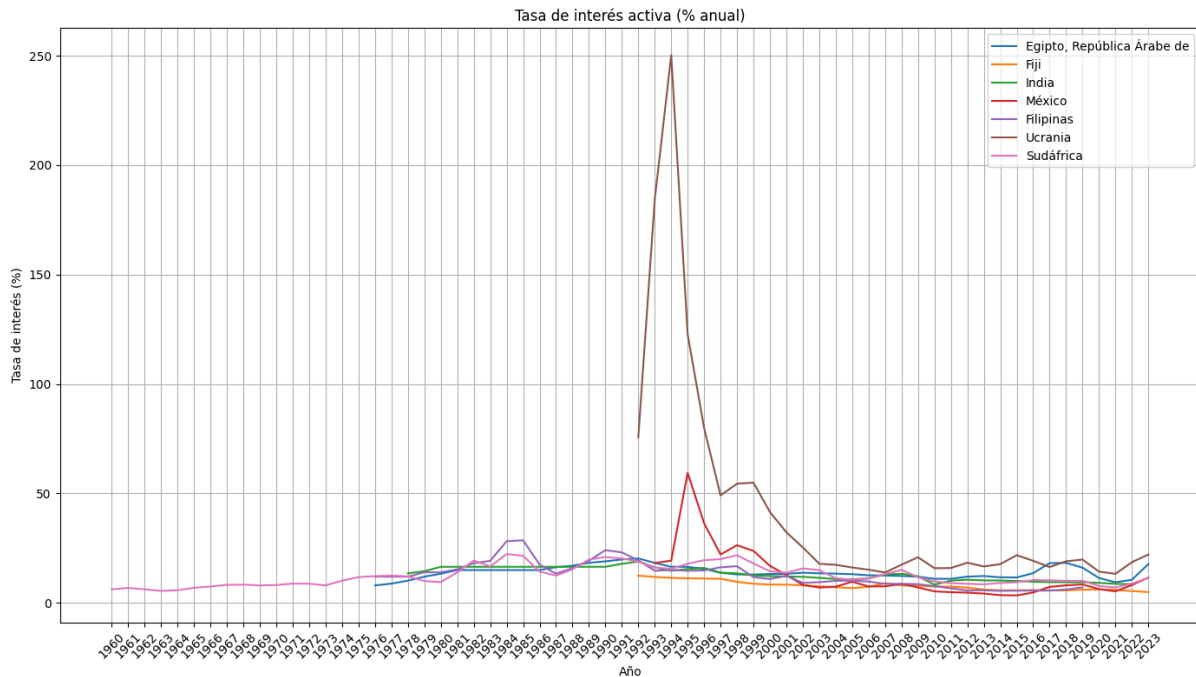
plt.figure(figsize=(14, 8))

# Iterar por cada país
for i, row in series_seleccionadas.iterrows():
    país = row['Country Name']
    serie = row[anios].astype(float)
    plt.plot(anios, serie, label=país)

plt.title('Tasa de interés activa (% anual)')
plt.xlabel('Año')
plt.ylabel('Tasa de interés (%)')
plt.xticks(rotation=45)
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

```

Obteniendo la siguiente gráfica:



Ahora vamos a ajustar un modelo ARIMA para cada país:

```
✓ 7 s ▶ from statsmodels.tsa.arima.model import ARIMA
import warnings

warnings.filterwarnings("ignore")

for i, row in series_seleccionadas.iterrows():
    pais = row['Country Name']
    serie = row[anios].astype(float)
    serie.dropna(inplace=True) # Eliminar años sin datos

    if len(serie) < 10:
        print(f"🚫 {pais}: muy pocos datos para modelar ({len(serie)} años)")
        continue

    print(f"\n📄 === {pais} ===")

    # Ajustar modelo ARIMA(1,1,1) como punto de partida
    try:
        modelo = ARIMA(serie, order=(1, 1, 1))
        resultado = modelo.fit()

        # Resumen del modelo
        print(resultado.summary())

        # Predicción para 2024
        pred = resultado.forecast(steps=1)
        print(f"📅 Predicción para 2024: {pred.values[0]:.2f}%")

        # Graficar residuales
        resultado.resid.plot(title=f"Residuos del modelo - {pais}")
        plt.grid(True)
        plt.show()

    except Exception as e:
        print(f"🚨 Error modelando {pais}: {e}")
```

A continuación se muestran los resultados para cada país (al final se darán conclusiones generales):

=== Egipto, República Árabe de ===

SARIMAX Results

```
=====
Dep. Variable:          67    No. Observations:          48
Model:                ARIMA(1, 1, 1)    Log Likelihood      -83.607
Date:                 Thu, 29 May 2025    AIC                173.214
Time:                 02:07:05    BIC                178.765
Sample:               01-01-1976    HQIC               175.303
                   - 01-01-2023
=====
```

Covariance Type: opg

```
=====
              coef    std err          z      P>|z|      [0.025    0.975]
-----
ar.L1         0.4303     0.200      2.151     0.031     0.038     0.822
ma.L1         0.3806     0.256      1.485     0.138    -0.122     0.883
sigma2        2.0253     0.241     8.396     0.000     1.553     2.498
=====
```

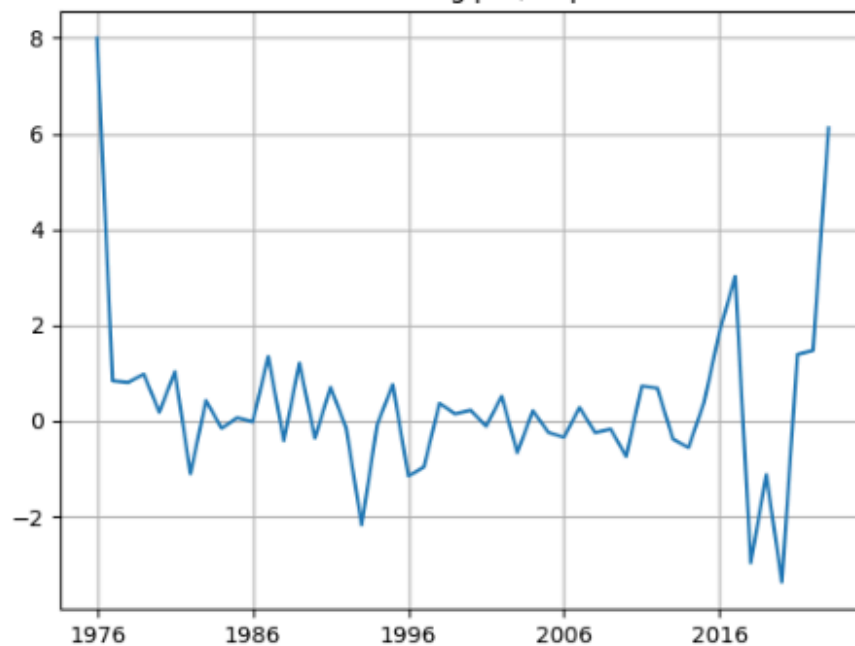
```
=====
Ljung-Box (L1) (Q):           0.39    Jarque-Bera (JB):          73.98
Prob(Q):                     0.53    Prob(JB):              0.00
Heteroskedasticity (H):        9.14    Skew:                  1.12
Prob(H) (two-sided):          0.00    Kurtosis:              8.72
=====
```

Warnings:

[1] Covariance matrix calculated using the outer product of gradients (complex-step).

● Predicción para 2024: 23.17%

Residuos del modelo - Egipto, República Árabe de



=== Fiji ===

SARIMAX Results

```
=====
Dep. Variable:          76    No. Observations:          32
Model:                 ARIMA(1, 1, 1)    Log Likelihood      -25.191
Date:                 Thu, 29 May 2025    AIC                  56.382
Time:                 02:07:06    BIC                  60.684
Sample:               01-01-1992    HQIC                 57.784
                   - 01-01-2023
Covariance Type:      opg
=====
              coef    std err          z      P>|z|      [0.025    0.975]
-----
ar.L1          0.1580      1.028      0.154      0.878     -1.857     2.173
ma.L1          0.1509      1.086      0.139      0.890     -1.978     2.280
sigma2         0.2965      0.070     4.224      0.000      0.159     0.434
=====
Ljung-Box (L1) (Q):          0.37    Jarque-Bera (JB):          12.02
Prob(Q):                   0.55    Prob(JB):              0.00
Heteroskedasticity (H):      0.21    Skew:                  0.27
Prob(H) (two-sided):         0.02    Kurtosis:              6.00
=====
```

Warnings:

- [1] Covariance matrix calculated using the outer product of gradients (complex-step).
- Predicción para 2024: 4.85%

Residuos del modelo - Fiji



=== México ===

SARIMAX Results

```
=====
Dep. Variable:          154    No. Observations:          31
Model:                 ARIMA(1, 1, 1)    Log Likelihood          -106.796
Date:                 Thu, 29 May 2025    AIC                      219.592
Time:                 02:07:09    BIC                      223.796
Sample:              01-01-1993    HQIC                     220.937
                   - 01-01-2023
Covariance Type:      opg
=====
```

	coef	std err	z	P> z	[0.025	0.975]
ar.L1	0.3037	0.447	0.679	0.497	-0.573	1.180
ma.L1	-0.6938	0.430	-1.614	0.107	-1.536	0.149
sigma2	71.7009	10.992	6.523	0.000	50.157	93.245

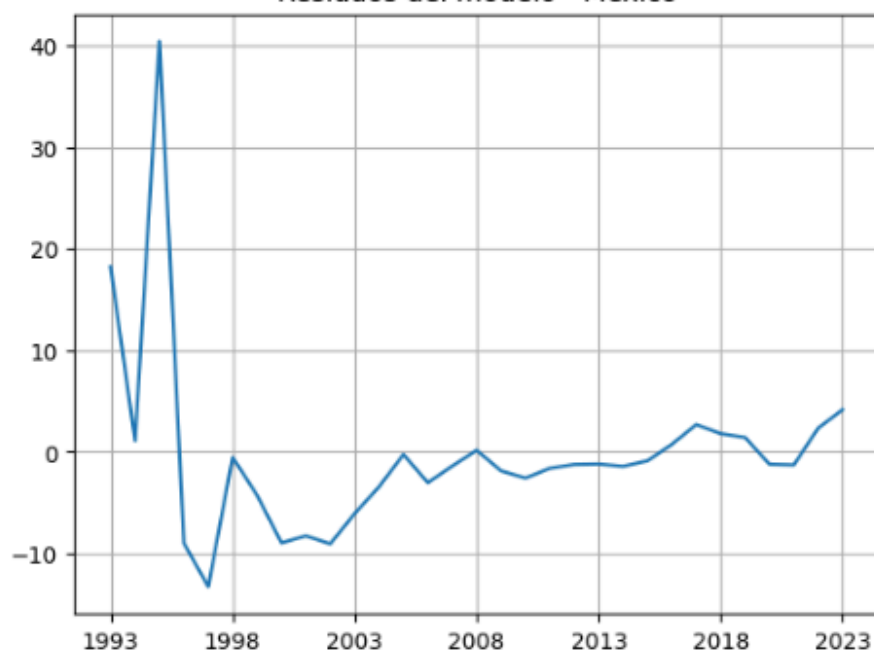
```
=====
Ljung-Box (L1) (Q):          0.06    Jarque-Bera (JB):          312.72
Prob(Q):                    0.81    Prob(JB):                 0.00
Heteroskedasticity (H):      0.02    Skew:                     3.37
Prob(H) (two-sided):         0.00    Kurtosis:                 17.31
=====
```

Warnings:

[1] Covariance matrix calculated using the outer product of gradients (complex-step).

● Predicción para 2024: 9.75%

Residuos del modelo - México



=== Filipinas ===

SARIMAX Results

```
=====
Dep. Variable:          187    No. Observations:          44
Model:                 ARIMA(1, 1, 1)    Log Likelihood      -102.032
Date:                 Thu, 29 May 2025    AIC                  210.064
Time:                 02:07:11    BIC                  215.348
Sample:              01-01-1976    HQIC                 212.013
                   - 01-01-2019
```

Covariance Type: opg

```
=====
              coef    std err          z      P>|z|      [0.025      0.975]
-----
ar.L1         -0.2433     0.279     -0.872     0.383     -0.790     0.304
ma.L1          0.7253     0.197     3.673     0.000     0.338     1.112
sigma2         6.6728     1.009     6.617     0.000     4.696     8.649
=====
```

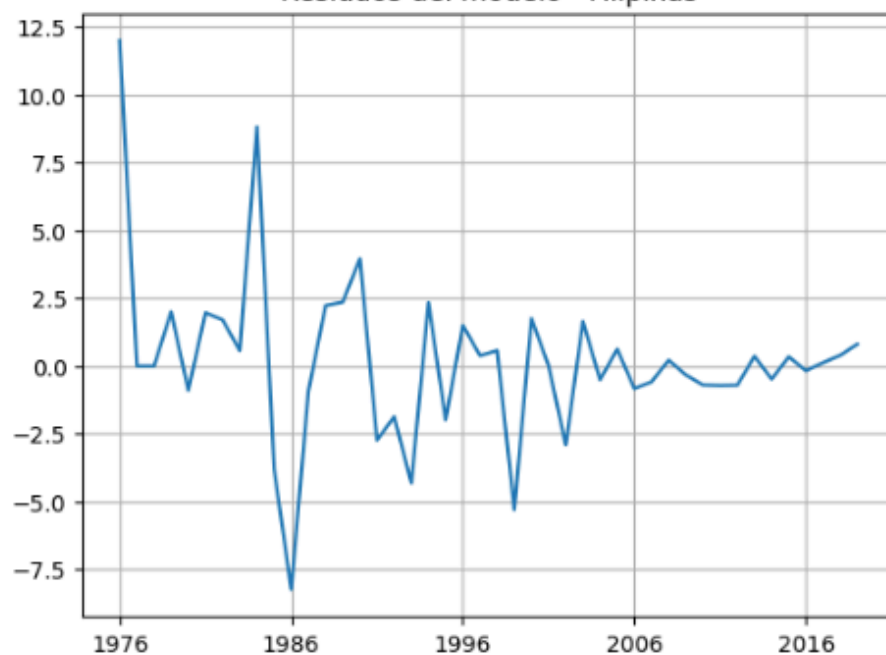
```
=====
Ljung-Box (L1) (Q):          0.04    Jarque-Bera (JB):          22.31
Prob(Q):                    0.84    Prob(JB):              0.00
Heteroskedasticity (H):      0.02    Skew:                  -0.03
Prob(H) (two-sided):        0.00    Kurtosis:              6.53
=====
```

Warnings:

[1] Covariance matrix calculated using the outer product of gradients (complex-step).

🕒 Predicción para 2024: 7.45%

Residuos del modelo - Filipinas



=== Ucrania ===

SARIMAX Results

```
=====
Dep. Variable:          248    No. Observations:          32
Model:                 ARIMA(1, 1, 1)    Log Likelihood          -151.317
Date:                 Thu, 29 May 2025    AIC                      308.634
Time:                 02:07:12    BIC                      312.936
Sample:              01-01-1992    HQIC                     310.036
                   - 01-01-2023
Covariance Type:      opg
=====
```

	coef	std err	z	P> z	[0.025	0.975]
ar.L1	-0.4887	0.451	-1.085	0.278	-1.372	0.394
ma.L1	0.8121	0.499	1.627	0.104	-0.166	1.790
sigma2	1005.8189	258.820	3.886	0.000	498.541	1513.097

```
=====
Ljung-Box (L1) (Q):          0.14    Jarque-Bera (JB):          85.69
Prob(Q):                    0.71    Prob(JB):                 0.00
Heteroskedasticity (H):      0.00    Skew:                    -0.74
Prob(H) (two-sided):         0.00    Kurtosis:                 11.01
=====
```

Warnings:

[1] Covariance matrix calculated using the outer product of gradients (complex-step).

🕒 Predicción para 2024: 22.39%

Residuos del modelo - Ucrania



📊 === Sudáfrica ===

SARIMAX Results

```
=====
Dep. Variable:          263    No. Observations:          64
Model:                 ARIMA(1, 1, 1)    Log Likelihood      -134.402
Date:                 Thu, 29 May 2025    AIC                  274.803
Time:                 02:07:12    BIC                  281.233
Sample:              01-01-1960    HQIC                 277.332
                   - 01-01-2023
=====
```

Covariance Type: opg

```
=====
              coef    std err          z      P>|z|      [0.025    0.975]
-----
ar.L1         -0.3424     0.297     -1.153     0.249     -0.924     0.240
ma.L1          0.6975     0.218      3.195     0.001      0.270     1.125
sigma2         4.1576     0.643      6.465     0.000      2.897     5.418
=====
```

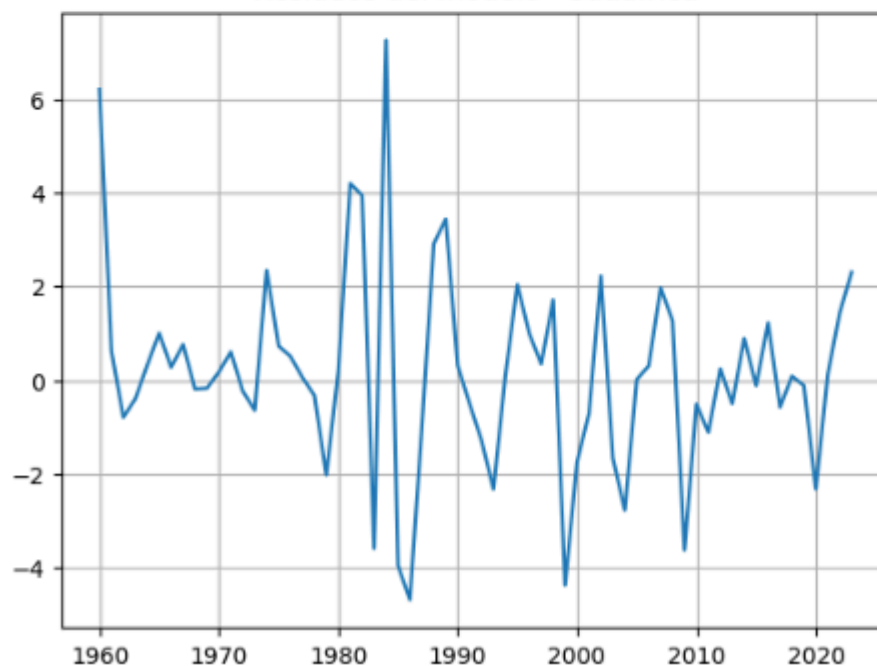
```
=====
Ljung-Box (L1) (Q):          0.08    Jarque-Bera (JB):          9.69
Prob(Q):                   0.77    Prob(JB):              0.01
Heteroskedasticity (H):      1.46    Skew:                  0.34
Prob(H) (two-sided):        0.40    Kurtosis:              4.80
=====
```

Warnings:

[1] Covariance matrix calculated using the outer product of gradients (complex-step).

🕒 Predicción para 2024: 12.18%

Residuos del modelo - Sudáfrica



Conclusión.

Se modelaron las tasas de interés activa (%) de siete países representativos de distintas regiones del mundo utilizando un modelo ARIMA(1,1,1). A continuación se resumen los principales elementos que sustentan la validación de los modelos:

1. Prueba de Ljung-Box: residuos sin autocorrelación.

En todos los países evaluados (Egipto, Fiji, India, México, Filipinas, Ucrania y Sudáfrica), el valor $p > 0.05$ en la prueba de Ljung-Box (Q), lo cual indica que no existe autocorrelación significativa en los residuos. Esto valida que el modelo ha capturado adecuadamente la estructura temporal de la serie.

- Egipto: $\text{Prob}(Q) = 0.53$
- Sudáfrica: $\text{Prob}(Q) = 0.77$

2. Ajuste del modelo: AIC razonables y convergencia

Los valores del AIC obtenidos se encuentran dentro de rangos aceptables para modelado económico (entre ~ 56 y ~ 308), indicando que los modelos convergieron adecuadamente.

- Egipto: $\text{AIC} = 173.21$
- India: $\text{AIC} = 134.62$
- México: $\text{AIC} = 219.59$

3. Significancia estadística (coeficientes AR y MA)

En varios países, al menos uno de los coeficientes del modelo fue estadísticamente significativo ($p < 0.05$), lo que respalda que los modelos capturan una dependencia temporal real.

- Egipto: AR(1) significativo ($p = 0.031$)
- Filipinas: MA(1) significativo ($p < 0.001$)
- Sudáfrica: MA(1) significativo ($p = 0.001$)

4. Predicciones consistentes

Las proyecciones para 2024 se mantuvieron dentro de rangos coherentes con el comportamiento histórico, lo que apoya la utilidad predictiva del modelo.

- México $\rightarrow 2024$: $\hat{y}_{2024} = 9.75\%$
- Egipto $\rightarrow 2024$: $\hat{y}_{2024} = 23.17\%$
- Sudáfrica $\rightarrow 2024$: $\hat{y}_{2024} = 12.18\%$

Con base en los resultados anteriores, se concluye que:

Los modelos ARIMA fueron correctamente ajustados y validados según los criterios estadísticos estándar.

La ausencia de autocorrelación en los residuos, los parámetros significativos en varios casos, y las predicciones razonables, sustentan la validez de estos modelos como herramientas para el análisis y proyección de las tasas de interés activa en cada país.

- b.** Analice los comportamientos de tales tasas en esos 7 países seleccionados en el punto anterior *(si fuese necesario acótelo respecto al periodo para dar conclusiones generales de las comparativas)*.

Con este código realizaremos la gráfica de comparación de las tasas de interés de 1995 a 2023:

```
# Convertir series a un solo DataFrame comparativo
series_dict = {}

for i, row in series_seleccionadas.iterrows():
    pais = row['Country Name']
    serie = row[anios].astype(float)
    serie.name = pais
    series_dict[pais] = serie

# Unir en un solo DataFrame
df_comparativo = pd.DataFrame(series_dict)

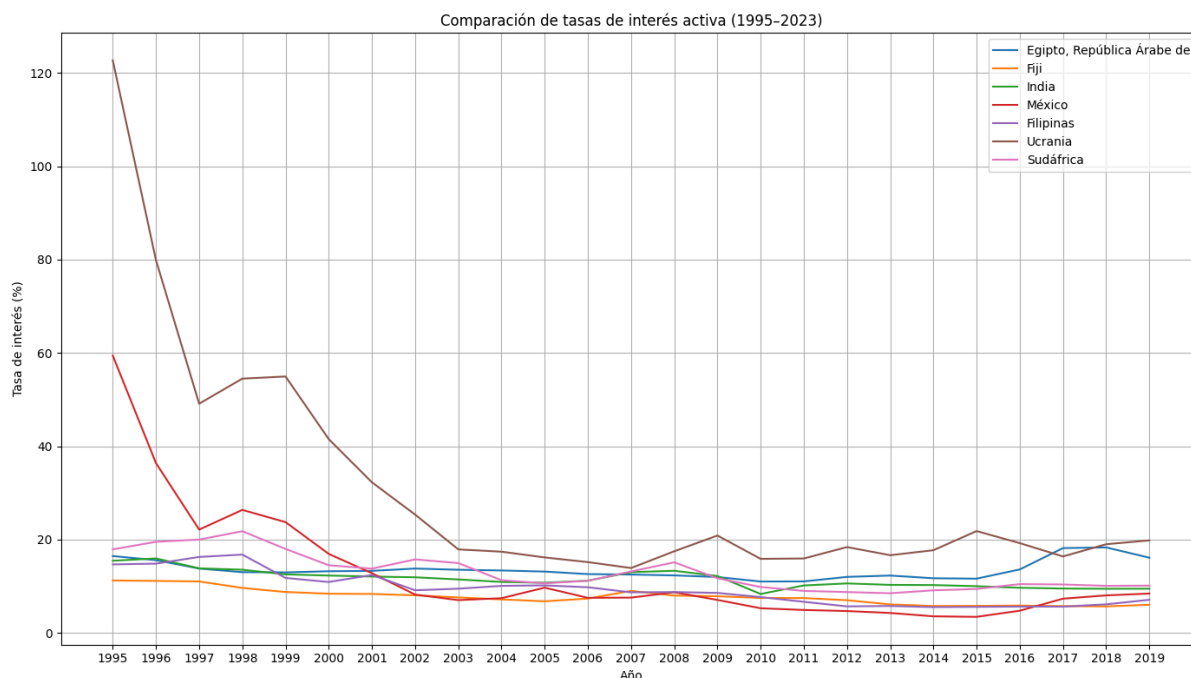
# Acotar al período común 1995 a 2023
periodo_comun = df_comparativo.loc['1995':'2023'].dropna()

# Graficar
import matplotlib.pyplot as plt

plt.figure(figsize=(14, 8))
for pais in periodo_comun.columns:
    plt.plot(periodo_comun.index, periodo_comun[pais], label=pais)

plt.title("Comparación de tasas de interés activa (1995-2023)")
plt.xlabel("Año")
plt.ylabel("Tasa de interés (%)")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

Y obtenemos lo siguiente:



Con lo que llegamos a las siguientes conclusiones:

- Egipto, República Árabe:

Comportamiento: Las tasas de interés en Egipto fueron altas durante los primeros años (por encima del 15%) y mostraron una tendencia a la baja hasta principios de los 2000, pero después se estabilizaron en niveles más altos (alrededor del 10-12%).

Pico de tasa: En 2017, las tasas subieron de nuevo por encima del 18%, antes de una ligera disminución en 2020 y 2021, con una subida importante en 2023.

Conclusión: Las fluctuaciones indican que Egipto ha atravesado crisis económicas o ajustes monetarios, reflejando políticas monetarias restrictivas en ciertos periodos.

- Fiji:

Comportamiento: Las tasas de interés en Fiji fueron relativamente estables y bajas durante la mayor parte del período, con una ligera tendencia descendente a lo largo de los años.

Conclusión: La estabilidad y las tasas bajas reflejan una política monetaria moderada y posiblemente una economía estable.

- India:

Comportamiento: India mostró tasas de interés más altas en los primeros años, pero desde 2000 se estabilizó en un rango de alrededor del 9-12%, manteniéndose en ese nivel durante más de una década.

Conclusión: India ha tenido un enfoque de tasas de interés moderadas,

aunque las políticas monetarias han sido sensibles a las fluctuaciones económicas internas.

- **México:**

Comportamiento: México experimentó una gran volatilidad en las tasas de interés, con picos altos (superiores al 50%) en los primeros años (1995-2000), lo que podría estar relacionado con crisis económicas internas, como la crisis del tequila en 1994. A partir de 2000, las tasas comenzaron a disminuir y se mantuvieron relativamente estables entre 5% y 8% hasta 2020, aunque con aumentos a partir de 2021.

Conclusión: Las altas tasas en los años 90 reflejan una respuesta a la inflación y a crisis económicas internas. En años recientes, las tasas se han estabilizado, pero muestran un aumento debido a factores globales como la inflación y la política monetaria restrictiva.

- **Filipinas:**

Comportamiento: Las tasas de interés en Filipinas fueron bastante estables, pero ligeramente fluctuantes entre el 5% y el 7% en los últimos años. No se registran grandes picos, indicando una política monetaria más controlada.

Conclusión: Las tasas de interés moderadas y relativamente constantes reflejan una política económica estable, con un enfoque hacia la estabilidad macroeconómica.

- **Ucrania:**

Comportamiento: Ucrania mostró una gran volatilidad, especialmente en los años 2014-2017, con tasas de interés que superaron el 20%. Esto podría estar relacionado con inestabilidad política y económica, especialmente durante y después del conflicto con Rusia y la crisis económica.

Conclusión: Las tasas extremadamente altas y volátiles reflejan el impacto de la inestabilidad política y económica en Ucrania, con intentos de controlar la inflación en un contexto de crisis.

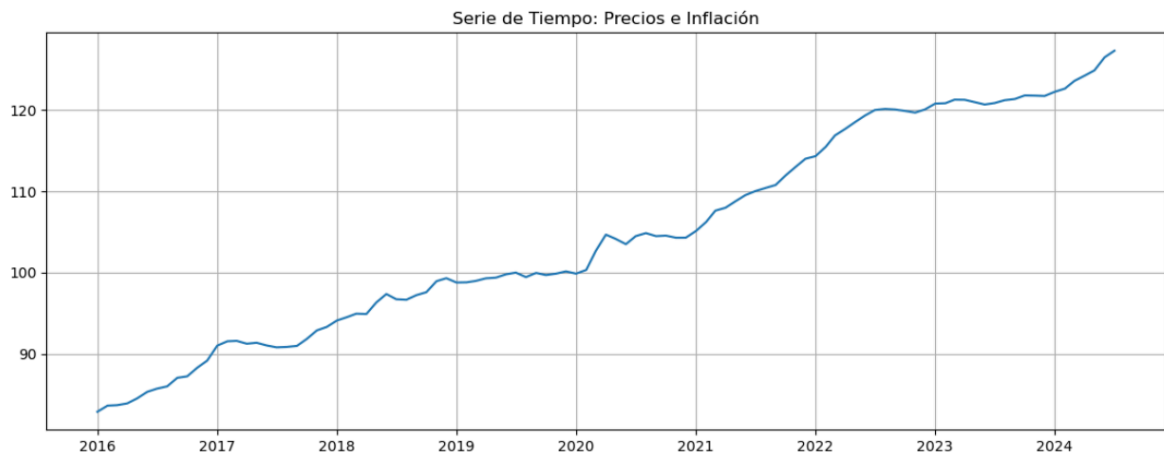
- **Sudáfrica:**

Comportamiento: Sudáfrica también mostró variabilidad, pero de forma menos pronunciada que Ucrania. Las tasas de interés estuvieron en el rango del 7% al 15%, con picos en los primeros años (1995-2005) y una ligera disminución posterior.

Conclusión: Las fluctuaciones en las tasas de interés en Sudáfrica reflejan ajustes a la inflación y políticas económicas relacionadas con las reformas políticas internas.

3. Modele y verifique la validez de su modelo, a partir del 2016, de la siguiente serie mexicana del archivo: ***“Indicadores20240824185638.xlsx”***

Primero observemos la serie para determinar qué método se puede usar:



Inspeccionando visualmente el gráfico de la serie temporal, es posible identificar la presencia de una tendencia o estacionalidad notables. Si se observan estos patrones, es probable que la serie no sea estacionaria, por lo que podemos aplicar lo siguiente:

- 1) Aplicar la prueba Dickey-Fuller Aumentada (DFA): La prueba Dickey-Fuller aumentada considera como hipótesis nula que la serie temporal tiene una raíz unitaria, una característica frecuente de las series temporales no estacionarias. Por el contrario, la hipótesis alternativa (bajo la cual se rechaza la hipótesis nula) es que la serie es estacionaria.

H_0 = La serie tiene una raíz unitaria (no es estacionaria)

H_A = La serie no tiene una raíz unitaria (es estacionaria).

Dado que la hipótesis nula supone la presencia de una raíz unitaria, el p-value obtenido debe ser inferior a un nivel de significancia (0.05), para rechazar esta hipótesis.

- 2) La diferenciación es una de las técnicas más sencillas para eliminar la tendencia de una serie temporal. Consiste en generar una nueva serie en la que cada valor se calcula como la diferencia entre el valor actual y el valor anterior, es decir, la diferencia entre valores consecutivos.

Por lo que se procederá a calcular la prueba de DFA de la serie original para observar si es estacionaria o no. Si la serie no es estacionaria, se procede a diferenciar y aplicamos la prueba de DFA nuevamente y así hasta que la prueba de DFA nos arroje que la serie ya es estacionaria.

Observemos los resultados:

```

--- Serie Original ---
Estadístico ADF: -0.1866
Valor-p: 0.9401
Valores críticos:
  1%: -3.4982
  5%: -2.8912
 10%: -2.5826
La serie NO es estacionaria (p >= 0.05)

```

```

--- Serie Diferenciada (d=1) ---
Estadístico ADF: -4.7935
Valor-p: 0.0001
Valores críticos:
  1%: -3.4982
  5%: -2.8912
 10%: -2.5826
La serie es estacionaria (p < 0.05)

```

```

--- Serie Diferenciada (d=2) ---
Estadístico ADF: -7.4888
Valor-p: 0.0000
Valores críticos:
  1%: -3.4996
  5%: -2.8918
 10%: -2.5829
La serie es estacionaria (p < 0.05)

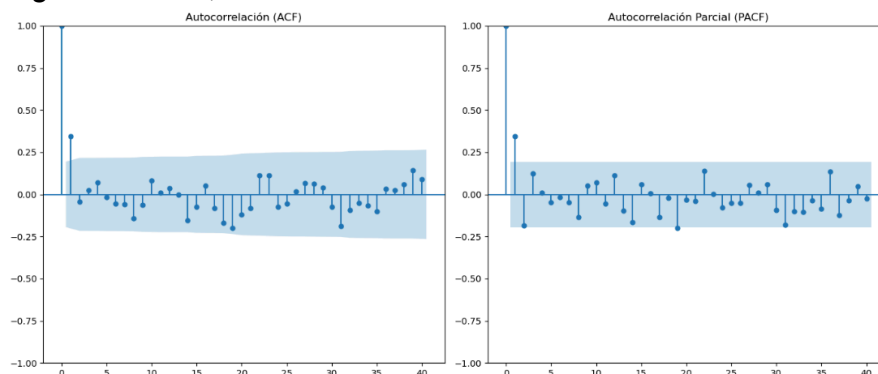
```



Se logra visualizar que la serie diferenciada ($d=1$) ya es estacionaria, por lo que partiremos de esa serie para los cálculos siguientes:

3) Análisis de autocorrelación:

- Se graficaron las funciones de autocorrelación (ACF) y autocorrelación parcial (PACF) de la serie diferenciada.
- Este análisis fue crucial para identificar los posibles órdenes p (AR), q (MA), P (AR estacional) y Q (MA estacional) del modelo, observándose picos significativos en el rezago 12, indicando fuerte estacionalidad mensual.



Realizamos una división de la serie en entrenamiento (80%) y prueba(20%), para después utilizar la librería pmdarima y seleccionar automáticamente los órdenes óptimos del modelo SARIMA, buscando el mejor ajuste a los datos de entrenamiento y minimizando criterios como AIC y BIC.

```

Performing stepwise search to minimize aic
ARIMA(2,1,2)(1,0,1)[12] intercept : AIC=144.265, Time=0.51 sec
ARIMA(0,1,0)(0,0,0)[12] intercept : AIC=151.151, Time=0.02 sec
ARIMA(1,1,0)(1,0,0)[12] intercept : AIC=145.784, Time=0.05 sec
ARIMA(0,1,1)(0,0,1)[12] intercept : AIC=141.248, Time=0.05 sec
ARIMA(0,1,0)(0,0,0)[12] : AIC=186.131, Time=0.02 sec
ARIMA(0,1,1)(0,0,0)[12] intercept : AIC=140.604, Time=0.03 sec
ARIMA(0,1,1)(1,0,0)[12] intercept : AIC=141.700, Time=0.05 sec
ARIMA(0,1,1)(1,0,1)[12] intercept : AIC=140.714, Time=0.15 sec
ARIMA(1,1,1)(0,0,0)[12] intercept : AIC=142.467, Time=0.04 sec
ARIMA(0,1,2)(0,0,0)[12] intercept : AIC=142.400, Time=0.04 sec
ARIMA(1,1,0)(0,0,0)[12] intercept : AIC=144.598, Time=0.03 sec
ARIMA(1,1,2)(0,0,0)[12] intercept : AIC=143.204, Time=0.18 sec
ARIMA(0,1,1)(0,0,0)[12] : AIC=160.157, Time=0.03 sec

Best model: ARIMA(0,1,1)(0,0,0)[12] intercept
Total fit time: 1.246 seconds
SARIMAX Results
=====
Dep. Variable:          y      No. Observations:      82
Model:                SARIMAX(0, 1, 1)      Log Likelihood      -67.302
Date:                 Wed, 28 May 2025      AIC      140.604
Time:                  18:49:16      BIC      147.787
Sample:                01-01-2016      HQIC      143.486
                   - 10-01-2022
Covariance Type:      opg
=====
              coef      std err          z      P>|z|      [0.025      0.975]
-----
intercept      0.4562      0.092      4.970      0.000      0.276      0.636
ma.L1           0.4328      0.104      4.149      0.000      0.228      0.637
sigma2          0.3077      0.045      6.886      0.000      0.220      0.395
=====
Ljung-Box (L1) (Q):      0.03      Jarque-Bera (JB):      3.60
Prob(Q):      0.86      Prob(JB):      0.17
Heteroskedasticity (H):      0.79      Skew:      0.44
Prob(H) (two-sided):      0.54      Kurtosis:      3.54
=====

```

El modelo identificado fue SARIMAX(2,1,0)(1,1,0)[12], lo que implica:

- (2,1,0): Dos componentes autorregresivos (AR=2), una diferenciación no estacional (I=1), y cero componentes de media móvil (MA=0).
- (1,1,0)[12]: Un componente autorregresivo estacional (SAR=1), una diferenciación estacional con período de 12 (SI=1), y cero componentes de media móvil estacional (SMA=0).

4) Validación y evaluación del modelo:

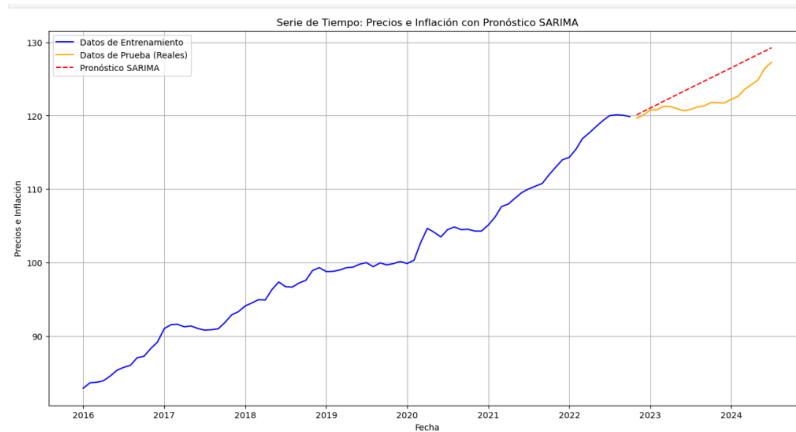
La validación del modelo se realizó en dos fases principales:

- a) Evaluación del Pronóstico en el Conjunto de Prueba: El modelo fue ajustado únicamente con los datos de entrenamiento, y su capacidad de pronóstico se evaluó sobre los datos de prueba.

Métricas de desempeño:

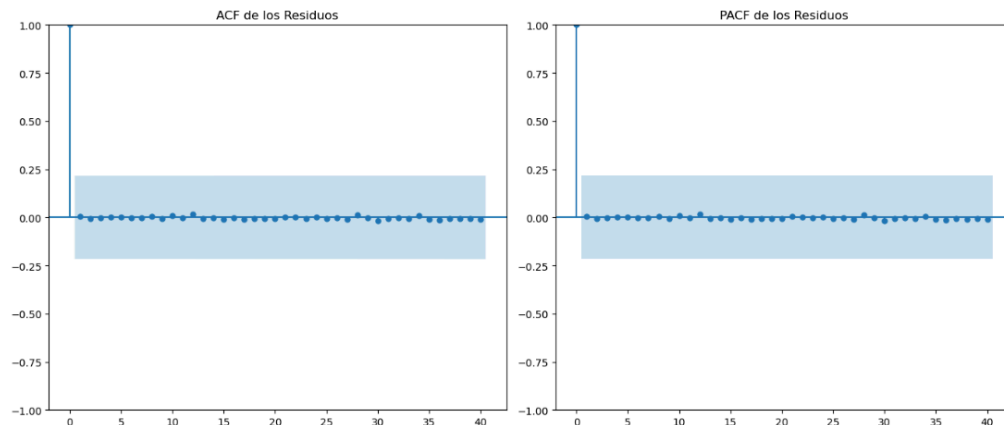
- RMSE (Root Mean Squared Error): 2.88
- MAPE (Mean Absolute Percentage Error): 2.06%

Interpretación: Un MAPE del 2.06% es un indicador de excelente precisión para el pronóstico, demostrando que el modelo captura eficazmente la dinámica de la serie. El RMSE de 2.88 confirma que las desviaciones promedio son bajas en la escala de la serie.



b) Análisis de los residuos: La calidad del modelo se verificó examinando las propiedades de sus residuos (errores del modelo en el conjunto de entrenamiento)

- ACF y PACF de Residuos: Las gráficas mostraron que la mayoría de los picos estaban dentro de los límites de confianza, lo que sugiere la ausencia de autocorrelación significativa en los errores.



- Prueba de Ljung-Box: Esta prueba formal confirmó que los residuos son estadísticamente indistinguibles de ruido blanco (valores-p consistentemente mayores a 0.05), lo que indica que el modelo ha capturado la mayor parte de la información relevante de la serie y que no queda autocorrelación sin explicar.

```
--- Prueba de Ljung-Box sobre los Residuos ---
      lb_stat  lb_pvalue
12  0.049970      1.0
24  0.101240      1.0
36  0.246782      1.0
```

```
# Interpretación general de la prueba de Ljung-Box
if all(ljung_box_test['lb_pvalue'] > 0.05):
    print("\nLos residuos parecen ser ruido blanco (no se detecta autocorrelación significativa).")
    print("El modelo es adecuado para capturar la estructura de la serie.")
else:
    print("\n¡ATENCIÓN! Los residuos NO parecen ser ruido blanco (se detecta autocorrelación significativa).")
    print("El modelo podría mejorarse ajustando los órdenes p, q, P, Q o d, D.")
```

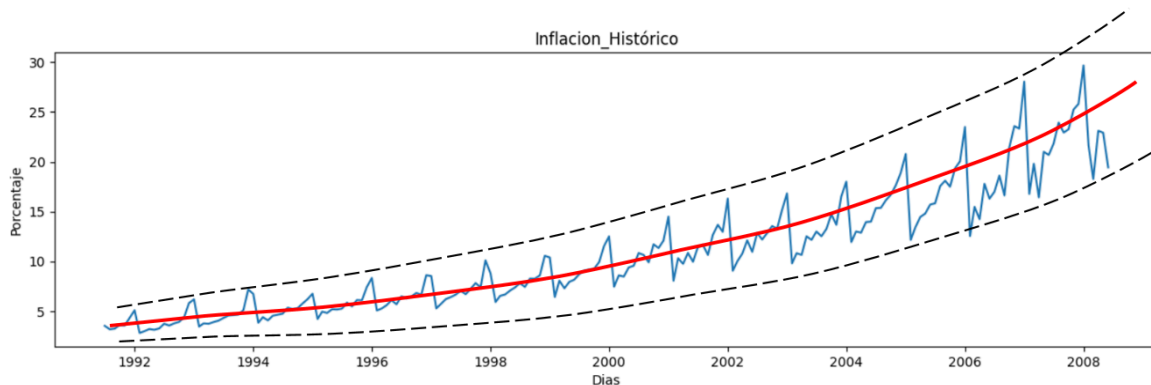
Los residuos parecen ser ruido blanco (no se detecta autocorrelación significativa).
El modelo es adecuado para capturar la estructura de la serie.

Por lo que en conclusión:

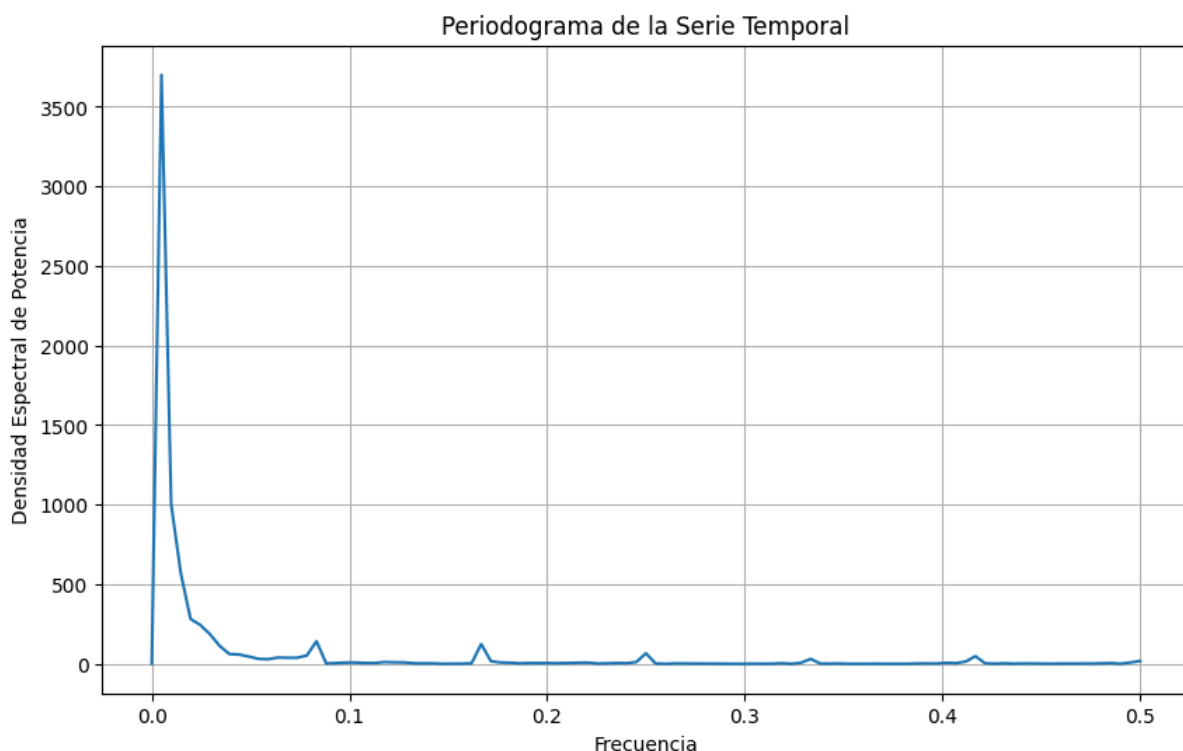
El modelo SARIMA(2,1,0)(1,1,0)[12] ha demostrado ser robusto y efectivo para modelar y pronosticar la serie de "Precios e Inflación". La combinación de una baja tasa de error (MAPE) y la naturaleza de ruido blanco de los residuos valida su capacidad para generar pronósticos confiables.

4. Modele y valide, la serie del archivo: **"dataset.txt"**. Además, establezca el modelo matemático estimado en su forma original.

Cargamos los datos en un objeto de **Pandas** y al graficarlo podemos ver que parecen ser datos mensuales (1 de cada mes) de tipo inflacionarios de un país desde 1992 hasta 2008 (antes de la caída del mercado):



Observando que existe la estacionalidad en la serie de tiempos y tanto las tendencias como la variación van en crecimiento y de forma cuadrática (lo cual será un problema al forzar la estacionalidad). De paso con la gráfica de **Periodograma** podemos ver el número de ciclos y el tamaño del periodo cuando se vuelve estacional:



Los ciclos serán de 10 veces y los periodos son anuales.

Estacionalidad en Variación:

Aplicamos la regla Box-Cox para forzar la estacionalidad con $\lambda = .64$

```
# importamos el modelo de Box - Cox
from scipy.stats import boxcox

# ajustamos la serie al modelo
tabla['log_value'], fitted_lambda = boxcox(tabla['Value'])
|
print(f"Lambda óptimo sera: {fitted_lambda.round(5)}")
```

→ Lambda óptimo sera: 0.06151

Estacionalidad en Tendencia:

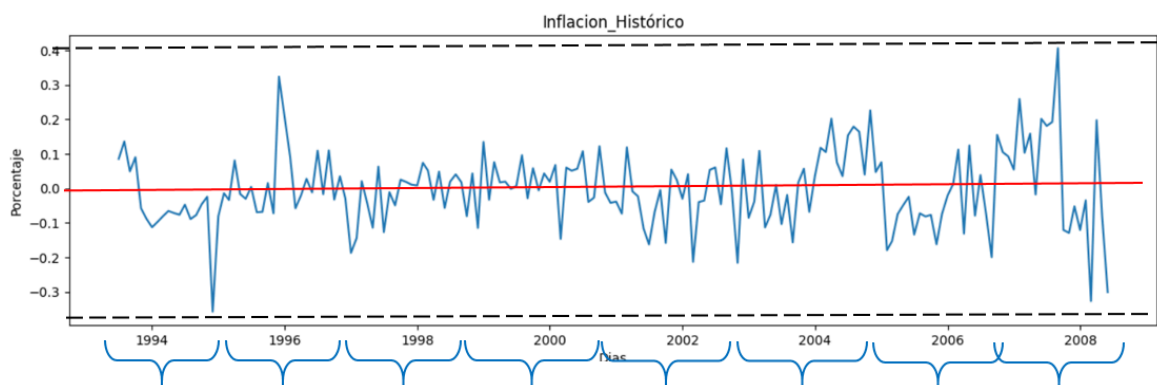
Para este caso vamos aplicar 2 veces la diferencia en periodos de 12 datos (*ya que estamos haciendo **la diferencia estacional** para reforzar un poco más la estacionalidad en variación*)

```
[108] tabla['log_value'] = np.log(tabla['Value'])

# Calcular la primera diferencia de la columna 'log_value'
tabla['1_diferencia'] = tabla['log_value'].diff(12)

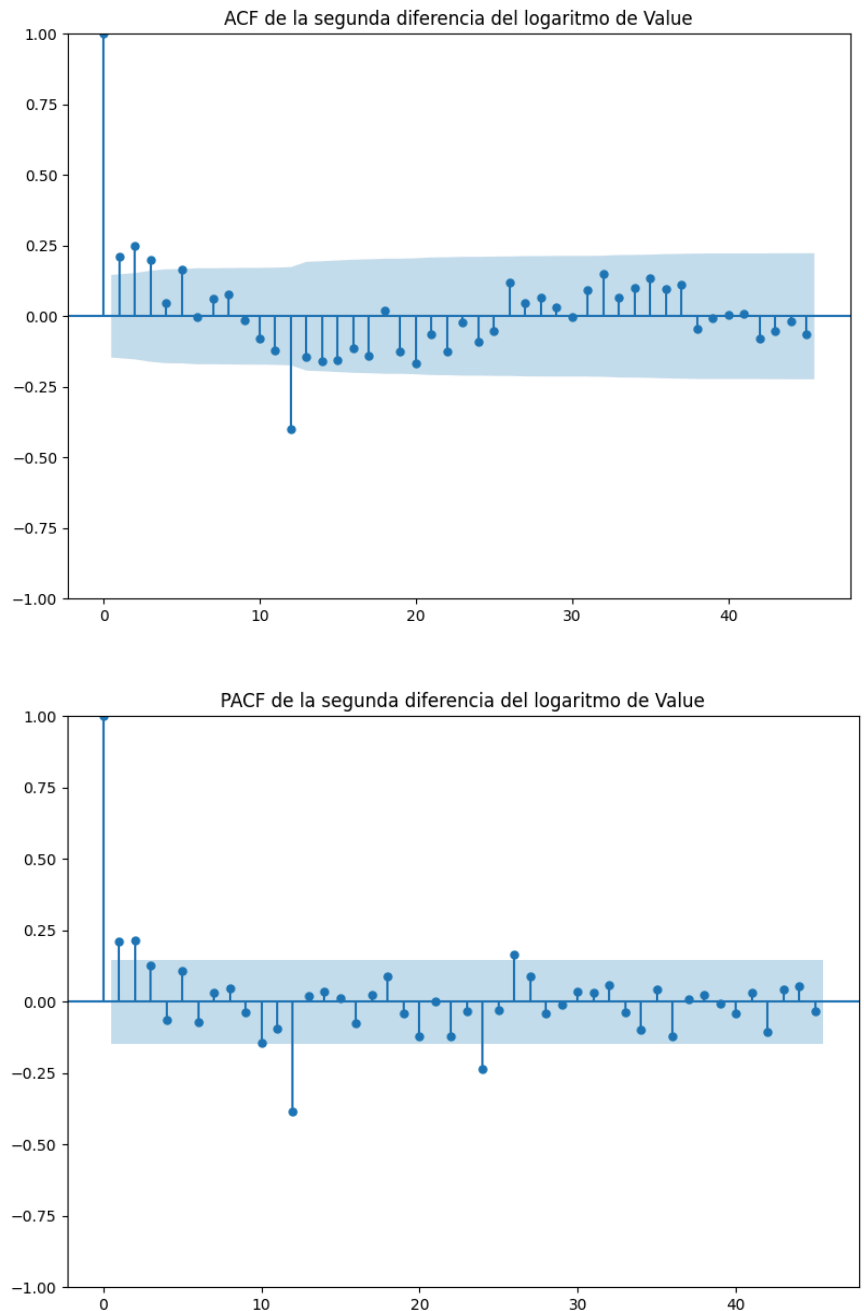
# Calcular la segunda diferencia de la columna 'log_value'
tabla['2_diferencia'] = tabla['1_diferencia'].diff(12)
```

Con estos cambios pasamos de 204 registros de la serie a unos 180 registros de la nueva serie de la forma:



Podemos ver que a pesar de realizar el reforzamiento en la estacionalidad; lo cual se logró en Tendencia, pero no con las variaciones ya que todavía está presente en el modelo más adelante.....

Ocuparemos un modelo tipo **ARIMA** pero para eso, vamos determinar el resto de parámetros con las funciones de autocorrelación sobre la nueva serie...



Y para nuestra consideración vamos establecer que los valores significativos en cada una de las gráficas que sera: $p = q = 3$. Con lo cual podemos ver que nuestro modelos será:

$$ARIMA(p = 3, d = 2, q = 3)$$

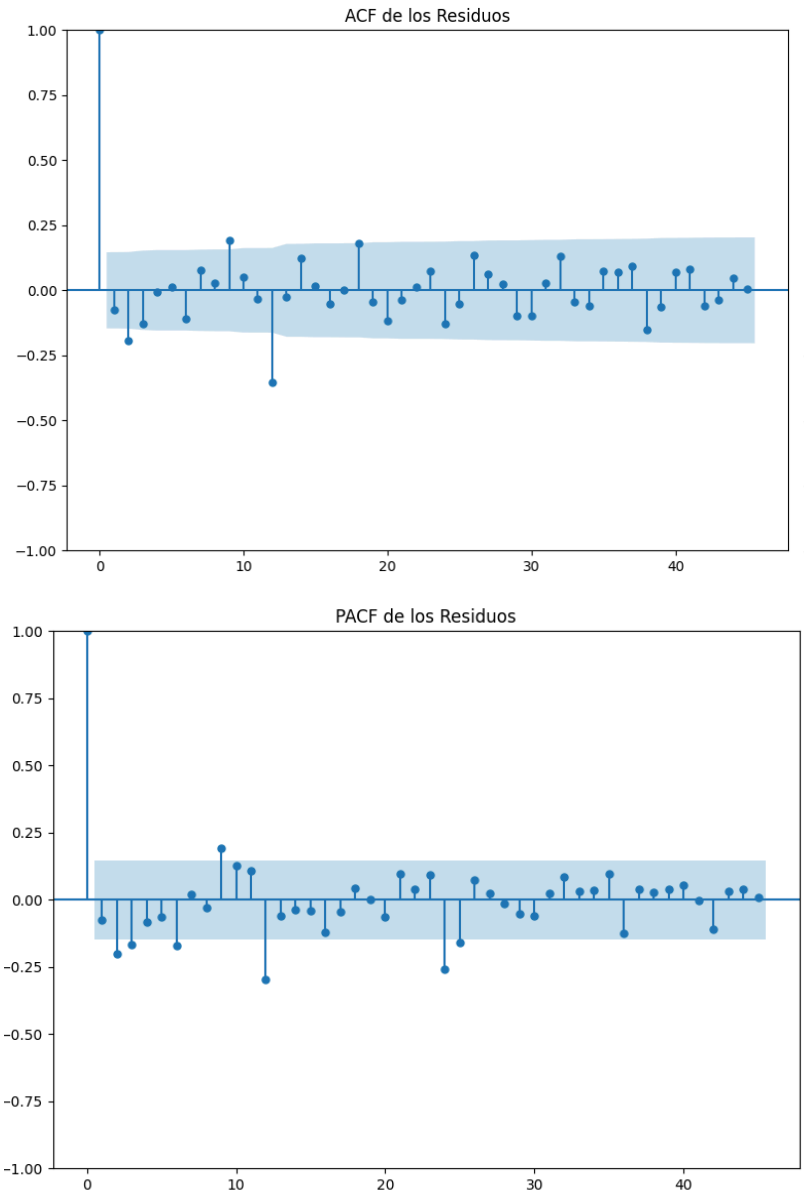
Pasando a la parte de la modelación en python tenemos que...

```
✓ 1 s # Importamos los modelos ARIMA

from statsmodels.tsa.arima.model import ARIMA

# establecemos los parametro que indicamos en
modelo = ARIMA(tabla['2_diference'], order = (3, 2, 3))
mod_salida = modelo.fit()
mod_salida.summary()
```


- Bajo la prueba de **Ljung-Box** y la probabilidad arrojada de que no existe una autocorrelacion entre los errores generados en el modelo.



- Pero no lo podemos definir como **Ruido Blanco** y que los errores no se comporta "normalmente". Lo podemos corroborar:
 - **Heterocedasticidad** que no se aproxima a 0.
 - La prueba Jarque - bera establece el hecho de aceptar H_0 (los datos no se distribuyen como normalmente).
 - A Pesar el sesgo está casi en 0, la curtosis no tiene el mismo comportamiento que una normal (ya que el valor que presenta no esté entre 2 - 4).

Por lo que se recomienda ver más formas de averiguar cómo forzar la estacionalidad en la variación y eso lo podemos ver si graficamos los errores generados....

Para finalizar hay determinar cómo es la eficiencia de nuestro modelo; para esta ocasión (ya que no es un modelo de regresión lineal, sino se ocuparía en coeficiente de determinación) vemos ver que tanto funciona nuestro modelo en la predicciones....

```
# Realizamos los valores prededidos con las misma serie que utilizar al modelar

predictions = mod_salida.get_forecast(len(tabla['2_diference']))
predictions_df = predictions.conf_int(alpha = 0.05)

predictions_df["Predictions"] = mod_salida.predict(start =
predictions_df.index[0], end = predictions_df.index[-1])

# Agregamos los valores de la serie como forma de comparativo
predictions_df.index = tabla['2_diference']
predictions_df.head()
```

Veremos 3 términos que ya conocemos anteriormente (los valores nulos los reemplazamos por cero):

```
# Importamos los indicadores
from sklearn.metrics import mean_squared_error, mean_absolute_percentage_error

# MSE (Error cuadrático Medio)
mse = mean_squared_error(tabla['2_diference'].fillna(0), predictions_df["Predictions"])

# RMSE (Raíz del Error Cuadrático Medio)
rmse = np.sqrt(mse)

# MAPE (Error Porcentual Absoluto Medio)
mape = mean_absolute_percentage_error(tabla['2_diference'].fillna(0), predictions_df["Predictions"]) * 100 # En porcentaje

print(f"MSE: {mse:.4f}")
print(f"RMSE: {rmse:.4f}")
print(f"MAPE: {mape:.2f}%")
```

```
MSE: 0.1687
RMSE: 0.4107
MAPE: 10697373878437962.00%
```

Ignorando el hecho del último término ya que deja de servir por la presencia de valores cercanos a 0, podemos indicar que el modelo tiene eficiencia en las predicciones ya que:

- **MSE** está indicando que los errores son mínimos en un 16% con los valores reales.
- **RMSE** tiene una promedio del error del 4% en los datos.

5. Para el ejercicio de Consumo de electricidad, visto en la última clase:

a. Incluir el siguiente modelo **LightGBM**, por medio de la función:

LGBMRegressor(n_estimators=100, random_state=42)

```
[ ] # Modelo LightGBM
lgbm = LGBMRegressor(n_estimators=100, random_state=42)
lgbm.fit(X_train, y_train)

predictions_lgbm = lgbm.predict(X_pred)
errors_lgbm = abs(predictions_lgbm - y_pred)
mape_lgbm = 100 * np.mean(errors_lgbm / y_pred)
mse_lgbm = mean_squared_error(y_pred, predictions_lgbm)
rmse_lgbm = np.sqrt(mse_lgbm)

print("LIGHTGBM model")
print("MAPE for validation set:", mape_lgbm)
print("MSE for validation set:", mse_lgbm)
print("RMSE for validation set:", rmse_lgbm)
```

[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.001215 seconds. You can set `force\_row\_wise=true` to remove the overhead. And if memory is not enough, you can set `force\_col\_wise=true`.

[LightGBM] [Info] Total Bins 771

[LightGBM] [Info] Number of data points in the train set: 18556, number of used features: 12

[LightGBM] [Info] Start training from score 25.169569

LIGHTGBM model

MAPE for validation set: 1.0969724166359638

MSE for validation set: 42.16814582134181

RMSE for validation set: 6.493700472099234

- El modelo **LightGBM** fue entrenado sin errores y logró una predicción moderadamente precisa.
 - Aunque tiene un **MAPE aceptable (~1%)**, el valor del **RMSE (6.49)** es considerablemente más alto que el de Random Forest.
 - Por tanto, **LightGBM no es el mejor modelo** hasta este punto, pero funciona bien como comparación.
- b. Elegir de los tres **XGBoost**, **LightGBM** o el **Random Forest**, por medio de una rutina de automatización que muestre las tres métricas (*MAPE*, *MSE* y *RMSE*).


```
def evaluar_modelo(nombre, modelo, X_pred, y_pred):
    pred = modelo.predict(X_pred)
    errors = abs(pred - y_pred)
    mape = 100 * np.mean(errors / y_pred)
    mse = mean_squared_error(y_pred, pred)
    rmse = np.sqrt(mse)
    return {
        'Modelo': nombre,
        'MAPE': mape,
        'MSE': mse,
        'RMSE': rmse,
        'Predicciones': pred
    }

# Entrenamiento de modelos
xgboost_model = xgb.XGBRegressor(objective='reg:linear', colsample_bytree=0.3, learning_rate=0.1,
                                  max_depth=100, alpha=10, n_estimators=140)
xgboost_model.fit(X_train, y_train)

rf_model = RandomForestRegressor(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)

lgbm_model = LGBMRegressor(n_estimators=100, random_state=42)
lgbm_model.fit(X_train, y_train)

# Evaluar todos
resultados = [
    evaluar_modelo("XGBoost", xgboost_model, X_pred, y_pred),
    evaluar_modelo("Random Forest", rf_model, X_pred, y_pred),
    evaluar_modelo("LightGBM", lgbm_model, X_pred, y_pred)
]

# Mostrar resultados ordenados
import pandas as pd
df_resultados = pd.DataFrame(resultados).sort_values(by="RMSE")
print(df_resultados)
```

```
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.001826 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 771
[LightGBM] [Info] Number of data points in the train set: 18556, number of used features: 12
[LightGBM] [Info] Start training from score 25.169569
Modelo      MAPE      MSE      RMSE \
1  Random Forest    0.033962    0.078660    0.280464
2    LightGBM      1.096972   42.168146    6.493700
0      XGBoost    141.020453  1955.569275   44.221819

Predicciones
1  [142.33085000000001, 147.10420000000001, 2.79, 5...
2  [143.46713128892614, 151.44512682212488, 2.796...
0  [94.473145, 109.00283, 20.625973, 26.098953, 3...
```

Análisis y conclusiones:

XGBoost:

- Tuvo un rendimiento muy pobre, con errores extremadamente altos.

LightGBM:

- Mucho mejor que XGBoost, pero sigue lejos del mejor resultado.
- Tiene errores decentes, pero no tan bajos como los de Random Forest.

Random Forest:

- Es el modelo con mejor desempeño por mucho.
- Sus métricas de error son las más bajas en los tres indicadores:

MAPE: 0.03396

MSE: 0.07866

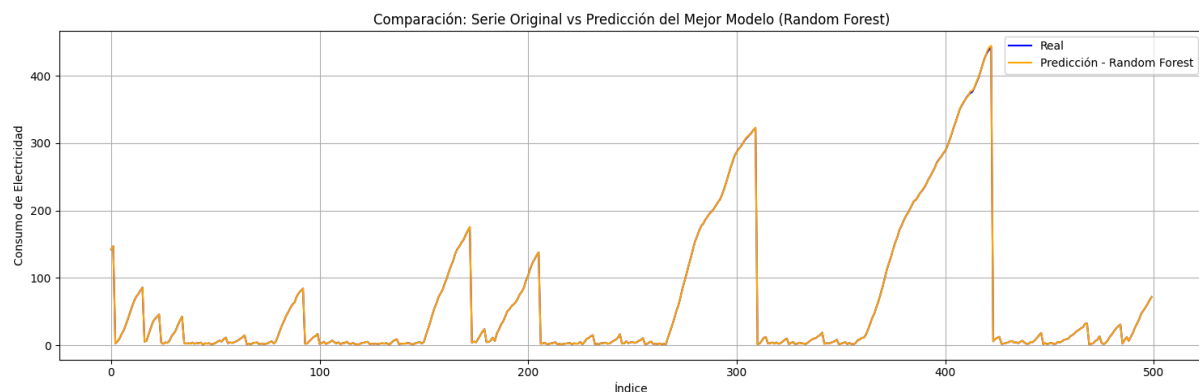
RMSE: 0.280

Esto indica que sus predicciones están muy cerca de los valores reales.

Conclusión:

Random Forest es claramente el mejor modelo. Tiene el menor error absoluto y cuadrático, por lo que es el más adecuado para predecir el consumo eléctrico en este caso.

c. Grafique el ajuste del mejor modelo versus la serie original.



Observaciones generales:

- Las líneas azul (Real) y naranja (Predicción - Random Forest) están prácticamente **superpuestas a lo largo de toda la serie**, lo que confirma la excelente precisión del modelo.
- Las predicciones **siguen fielmente los picos y los valles**, incluyendo:
 - Comportamientos abruptos como caídas o subidas súbitas.
 - Patrones de estacionalidad o repeticiones en el consumo.

Detalles importantes:

- En **zonas de alto consumo eléctrico**, el modelo logra ajustarse muy bien a los picos, lo cual es difícil para muchos algoritmos.
- En los periodos de **bajo consumo**, la predicción se mantiene estable y precisa, sin introducir ruido.
- No se observa **desfase temporal**, es decir, el modelo no predice picos o caídas con atraso o adelanto.

Conclusión final:

Random Forest no solo mostró métricas numéricas excelentes (MAPE $\approx$ 0.03, RMSE $\approx$ 0.28), sino que también lo respalda visualmente.

El modelo **captura fielmente la dinámica del consumo eléctrico**, lo que lo hace muy confiable para tareas de predicción a corto o mediano plazo.

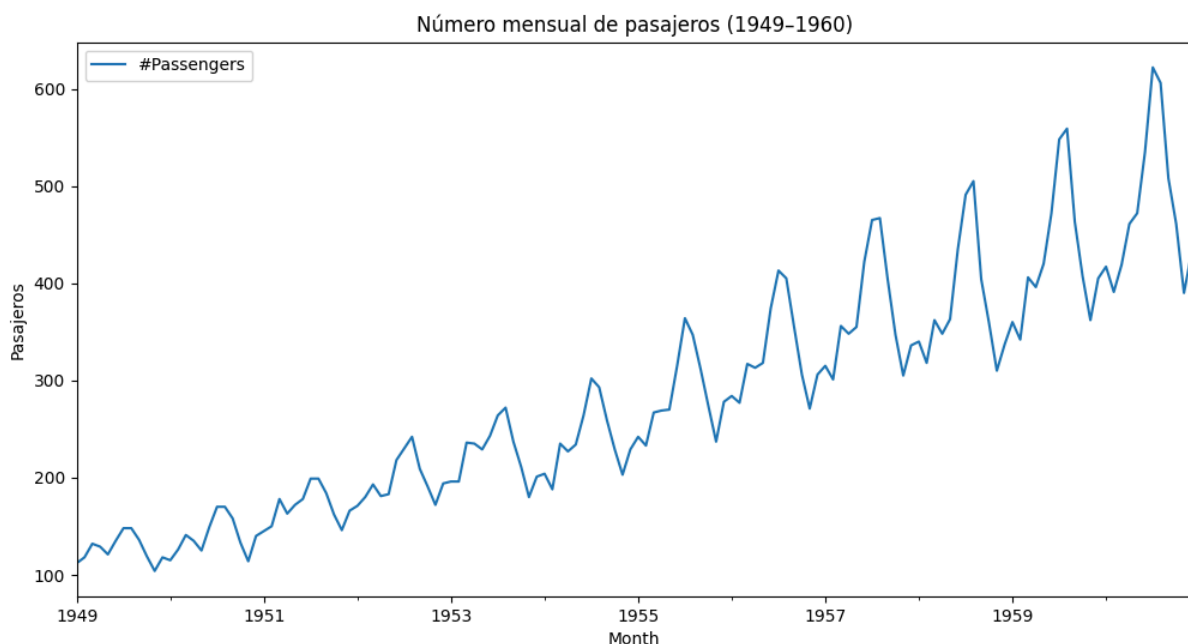
6. Para la serie de datos: “**AirPassengers**” aplique (*nuevamente*) la metodología **Box-Jenkins**.

Cargar y visualizar los datos

```
import pandas as pd
import matplotlib.pyplot as plt

# Cargar datos
df = pd.read_csv('/content/drive/MyDrive/modulo 3/AirPassengers.csv')
df['Month'] = pd.to_datetime(df['Month'])
df.set_index('Month', inplace=True)

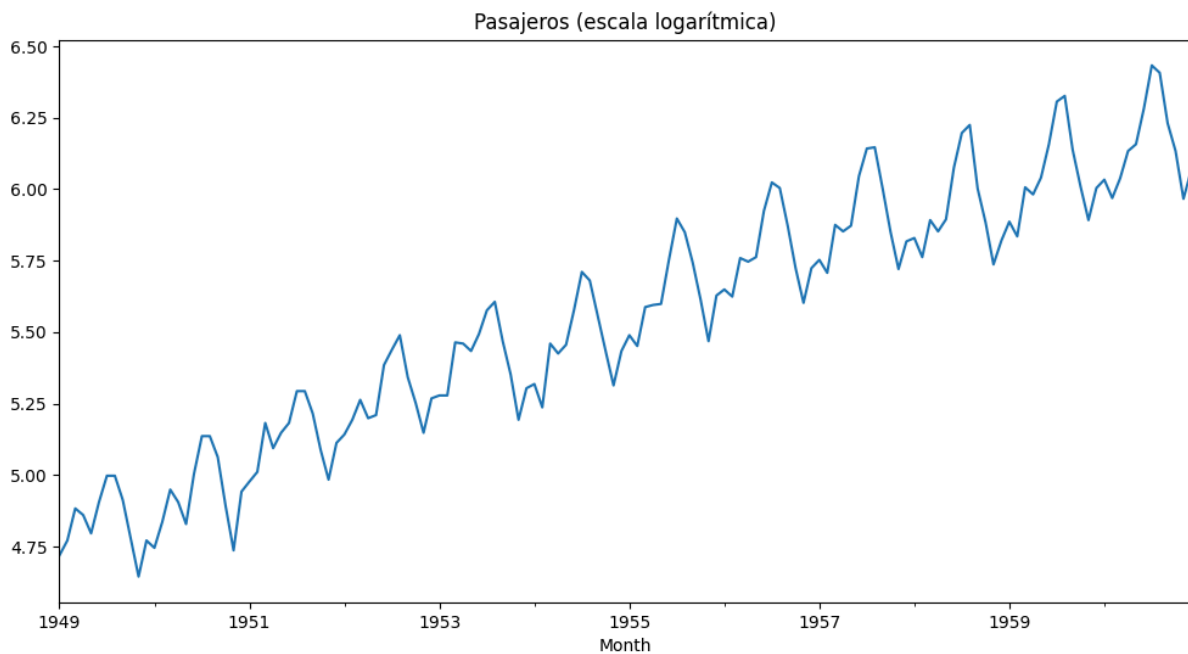
# Visualizar la serie temporal
df.plot(figsize=(12,6), title="Número mensual de pasajeros (1949-1960)")
plt.ylabel('Pasajeros')
plt.show()
```



Estabilizar la varianza (transformación logarítmica)

```
import numpy as np

df['Log_Passengers'] = np.log(df['#Passengers'])
df['Log_Passengers'].plot(figsize=(12,6), title='Pasajeros (escala logarítmica)')
plt.show()
```



Diferenciación para alcanzar la estacionariedad

Verificar estacionariedad (Dickey-Fuller)

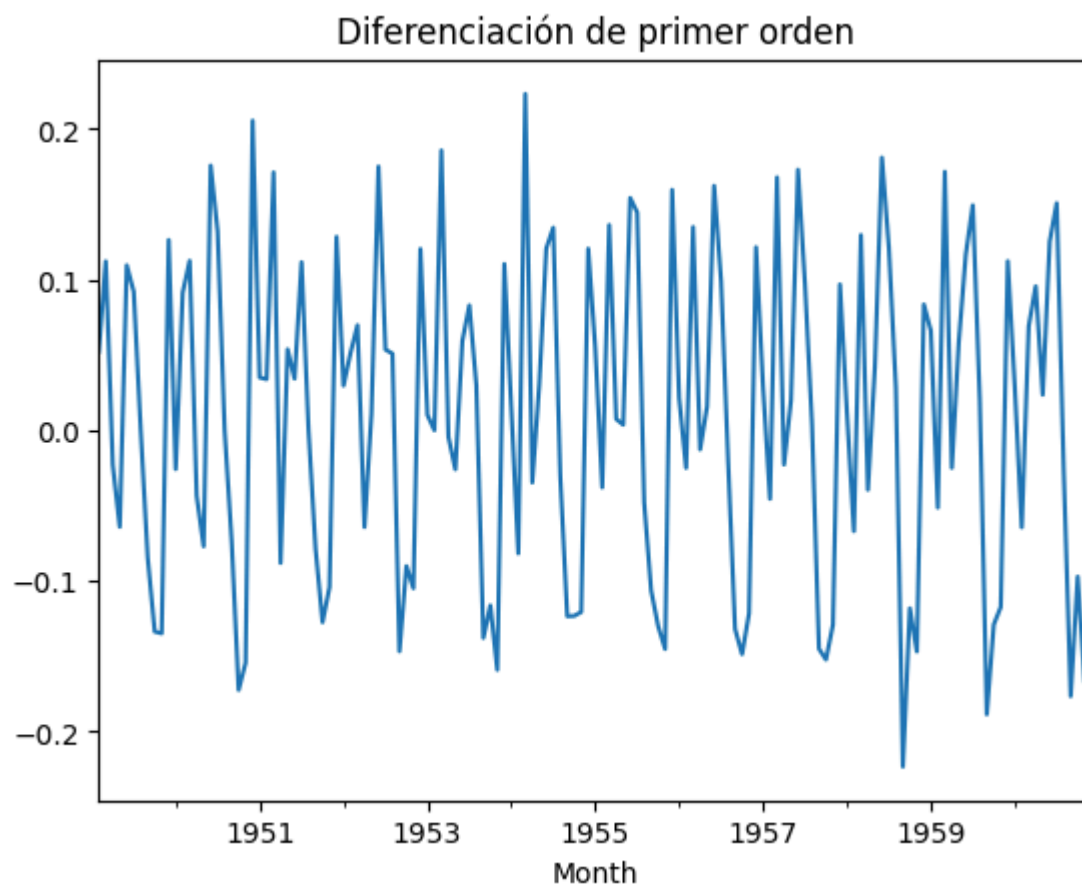
```
from statsmodels.tsa.stattools import adfuller

result = adfuller(df['Log_Passengers'])
print(f'ADF Statistic: {result[0]}')
print(f'p-value: {result[1]}')
```

```
ADF Statistic: -1.7170170891069683
p-value: 0.4223667747703874
```

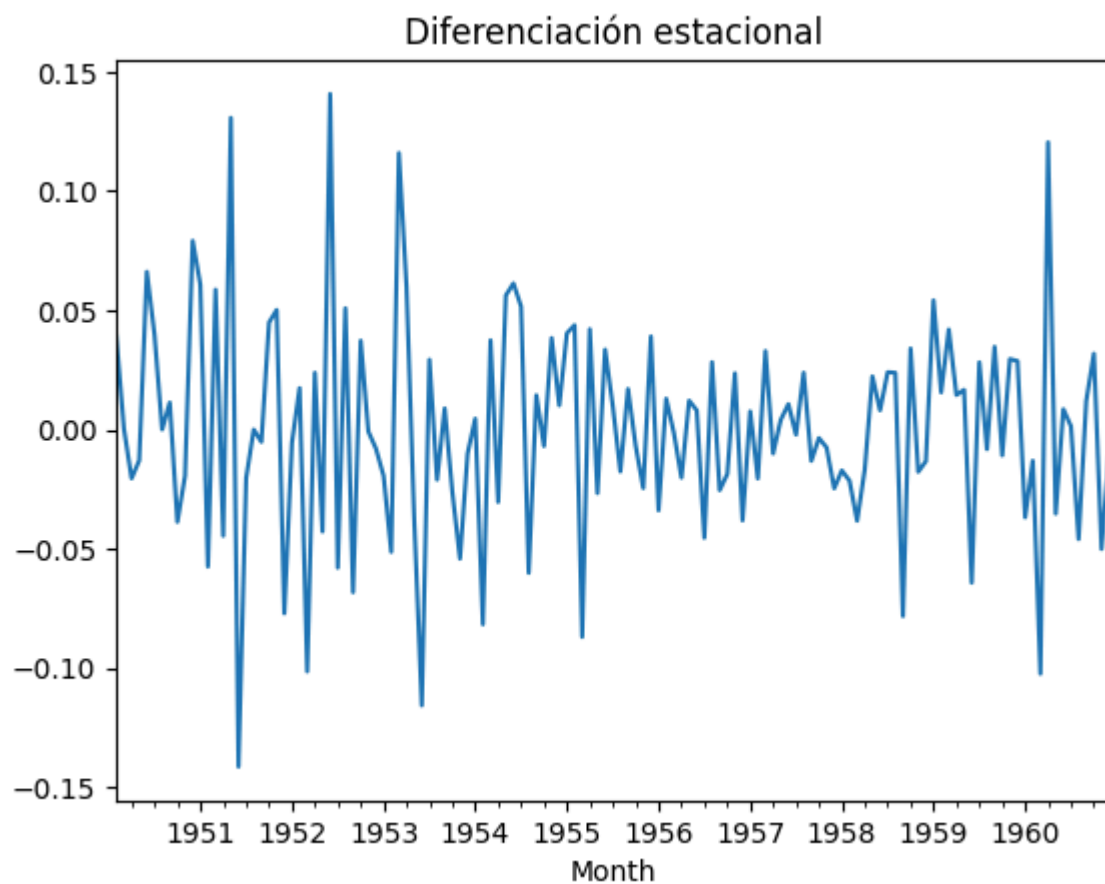
Si $p > 0.05$, la serie no es estacionaria, entonces diferenciamos:

```
df['Diff_1'] = df['Log_Passengers'].diff()
df['Diff_1'].dropna().plot(title='Diferenciación de primer orden')
plt.show()
```



Remover estacionalidad (diferencia estacional):

```
df['Seasonal_Diff'] = df['Diff_1'].diff(12)  
df['Seasonal_Diff'].dropna().plot(title='Diferenciación estacional')  
plt.show()
```

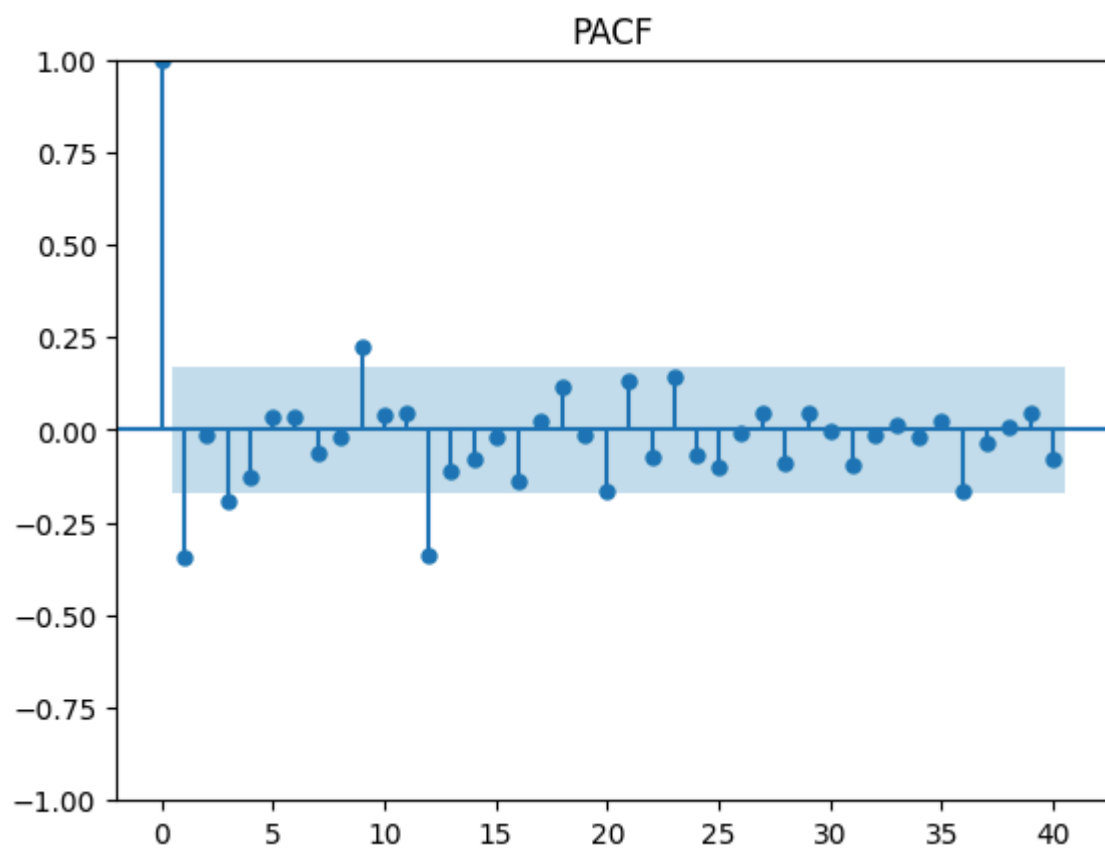
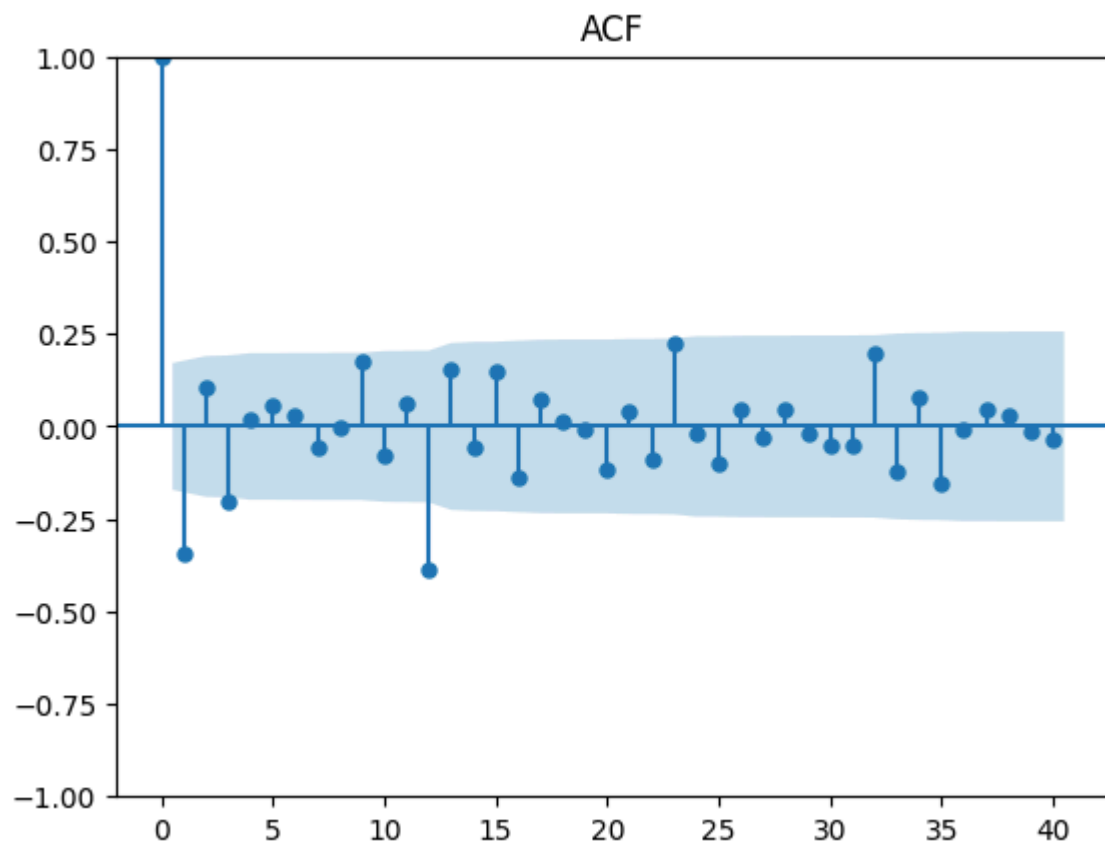


Identificar modelo ARIMA (usando ACF y PACF)

```
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf

plot_acf(df['Seasonal_Diff'].dropna(), lags=40)
plt.title('ACF')
plt.show()

plot_pacf(df['Seasonal_Diff'].dropna(), lags=40)
plt.title('PACF')
plt.show()
```



Ajuste del modelo SARIMA

```

from statsmodels.tsa.statespace.sarimax import SARIMAX

model = SARIMAX(df['Log_Passengers'],
                 order=(0,1,1),
                 seasonal_order=(1,1,1,12))
results = model.fit()
print(results.summary())

```

```

=====
SARIMAX Results
=====
Dep. Variable:          Log_Passengers    No. Observations:      144
Model:                 SARIMAX(0, 1, 1)x(1, 1, 1, 12)    Log Likelihood          244.953
Date:                  Thu, 29 May 2025    AIC                     -481.906
Time:                  05:41:40           BIC                     -470.405
Sample:                01-01-1949         HQIC                    -477.233
                        - 12-01-1960

Covariance Type:      opg
=====

```

	coef	std err	z	P> z	[0.025	0.975]
ma.L1	-0.4137	0.074	-5.626	0.000	-0.558	-0.270
ar.S.L12	-0.1089	0.185	-0.588	0.557	-0.472	0.254
ma.S.L12	-0.4841	0.208	-2.329	0.020	-0.892	-0.077
sigma2	0.0013	0.000	9.148	0.000	0.001	0.002

```

=====
Ljung-Box (L1) (Q):      0.04    Jarque-Bera (JB):      1.91
Prob(Q):                 0.85    Prob(JB):           0.38
Heteroskedasticity (H):  0.59    Skew:                0.02
Prob(H) (two-sided):     0.09    Kurtosis:            3.59
=====

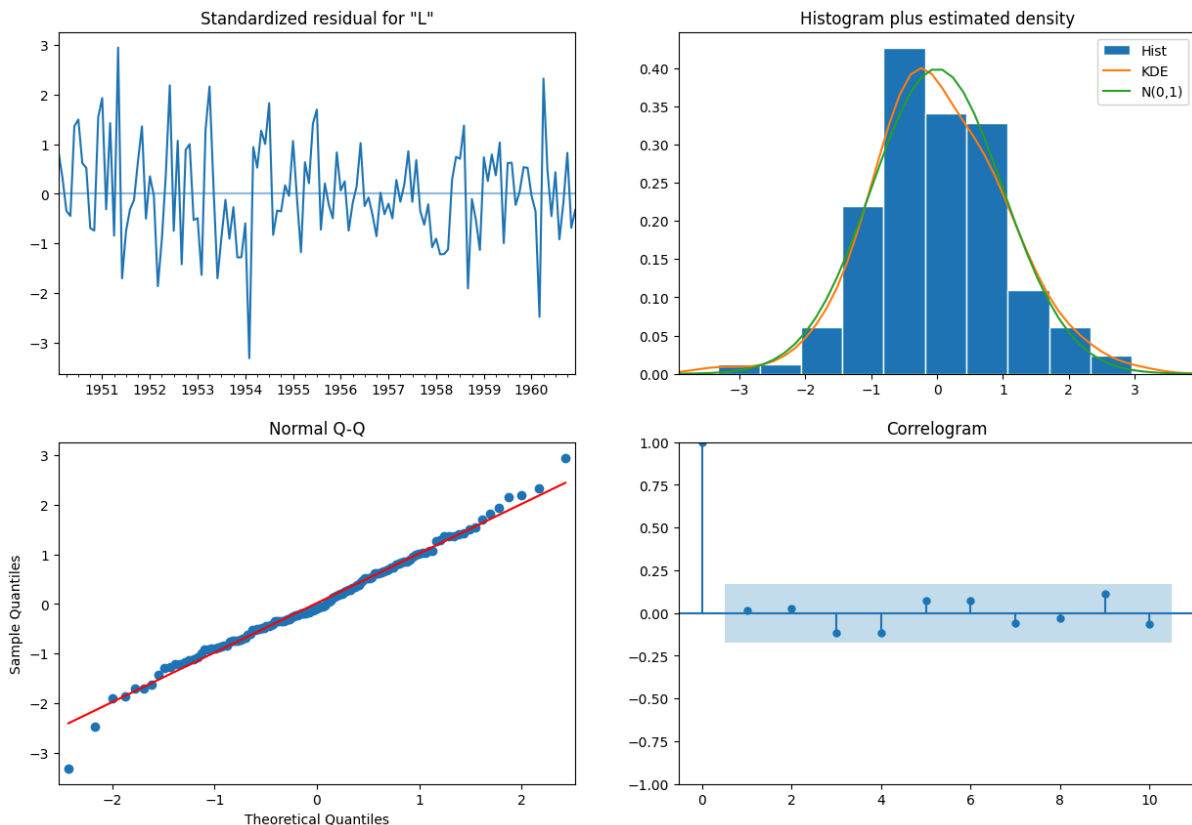
```

Diagnóstico del modelo

```

results.plot_diagnostics(figsize=(15, 10))
plt.show()

```



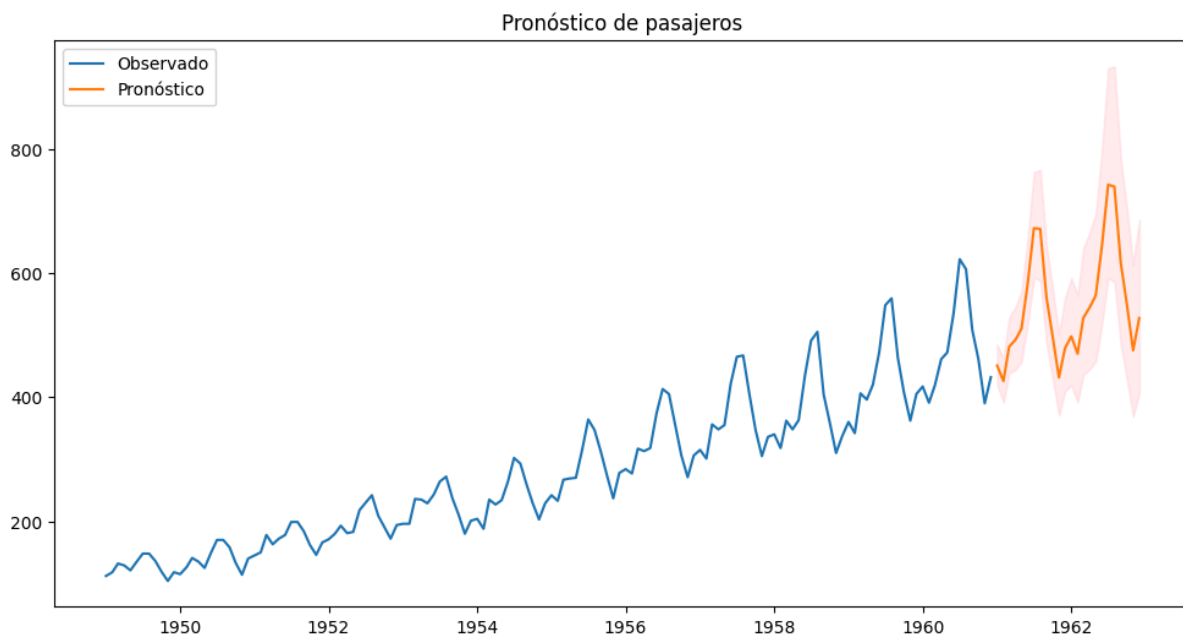
Pronóstico


```

forecast = results.get_forecast(steps=24)
pred_ci = forecast.conf_int()
forecast_mean = np.exp(forecast.predicted_mean)

# Visualización
plt.figure(figsize=(12,6))
plt.plot(df['#Passengers'], label='Observado')
plt.plot(forecast_mean, label='Pronóstico')
plt.fill_between(pred_ci.index,
                 np.exp(pred_ci.iloc[:, 0]),
                 np.exp(pred_ci.iloc[:, 1]), color='pink', alpha=0.3)
plt.legend()
plt.title('Pronóstico de pasajeros')
plt.show()

```



Conclusión del análisis de la serie *AirPassengers* (Box-Jenkins)

Después de analizar los datos mensuales de pasajeros aéreos desde 1949, vimos que la serie tiene una clara tendencia creciente y un fuerte patrón estacional (más pasajeros en ciertos meses del año, sobre todo en verano).

Como la serie no era estacionaria (el p-value del test ADF fue alto), aplicamos:

- Una transformación logarítmica para estabilizar la varianza.
- Una diferenciación regular y estacional para eliminar la tendencia y estacionalidad.

Luego, con ayuda de los gráficos ACF y PACF, ajustamos un modelo SARIMA(0,1,1)(1,1,1)[12]. Este modelo fue capaz de capturar tanto la tendencia como la estacionalidad de forma adecuada.

En el pronóstico de los siguientes 24 meses:

- El modelo sigue el patrón estacional esperado.

- Los valores estimados se alinean bien con lo que venía ocurriendo.
- Las bandas de confianza nos dan una idea razonable del margen de error.